

Intent-Governed Tool Authorization for AI Agents

Genliang Zhu

Accentrust

Georgia Institute of Technology

Chu Wang

Accentrust

University of Illinois Urbana-Champaign

Abstract

Tool-using AI agents commonly operate under integration credentials whose static permissions exceed a user’s current request. We present *Intent-Governed Access Control* (IGAC), a server-side authorization layer that converts a trusted user request into a short-lived intent certificate, monotonically filters the statically authorized tool manifest, and checks proposed tool and payload effects before execution. The model and certificate generator remain untrusted: IGAC cannot grant authority outside static policy, and request-level confinement is conditional on certificate and effect-bound fidelity. We formalize these guarantees, implement a reusable IGAC path over an OpenPort governance substrate, and evaluate the mechanism using endpoint tests, 176 runtime-backed synthetic tasks, real-model classifier and planner pilots, 306 end-to-end model-task runtime trials, and a 36-trial benchmark-shaped external subset. In the deterministic runtime comparison, author-reference IGAC reduces the archived composite exposure-or-path indicator from 1.0000 to 0, while both OpenPort-only and IGAC observe zero completed unsafe executions. In the end-to-end model runs, the combined IGAC–OpenPort path again observes zero completed unsafe executions, but unsafe accepted authority remains 0.0909–0.2727; strict benign completion is 0, while the governed-artifact rate is 0.25–0.50. Every residual unsafe acceptance is a non-executed draft. A trace-backed normalizer counterfactual removes this residual authority at substantial utility cost. These results support mechanism-level static-policy non-expansion and expose certificate precision, rather than model intent classification alone, as the principal remaining bottleneck. They do not establish production robustness or deployment-level safety improvement.

Keywords: AI agents, authorization, access control, least privilege, prompt injection.

I. INTRODUCTION

AI agents are moving from conversational assistance to operational action. A deployed agent can inspect records, update state, export data, coordinate workflows, and act across application boundaries. In such settings, tool access is no longer a convenience feature of a language model interface. It is an application security boundary.

The access-control question normally asked by an application gateway is credential-centric: does this token, app, or integration have permission to call this endpoint? Credential-centric authorization is necessary. It supports revocation, least privilege, tenant isolation, and accountability. But agentic tool use introduces a second question that traditional API authorization rarely models: *is the proposed tool call justified by what the user currently asked the agent to do?*

This distinction matters because the same credential can be legitimately broad while a particular user request is narrow. An agent may have both read and export scopes because the organization approves those capabilities for some workflows. Yet if the user asks “summarize recent records for this period,” exposing a bulk export tool to the model is unnecessary and potentially dangerous. If the model later proposes an export because an external document, retrieved note, or malicious tool description influenced it, credential-only authorization cannot distinguish the overbroad effect from the user’s original summary request. Likewise, a model may construct a plausible update payload that touches a different resource than the user named, or a delete call that exceeds the intended target set.

The recent security literature shows that this is not a hypothetical edge case. AgentDojo evaluates prompt-injection attacks and defenses in tool-using agents [1]. InjecAgent studies indirect prompt injection in tool-integrated agents and categorizes attacks into direct harm and data exfiltration [2]. ToolHijacker and ToolTweak show that tool descriptions and names can bias or hijack tool selection [3], [4]. MCP-oriented work identifies tool poisoning, shadowing, and protocol-specific semantic attacks [5]–[8]. These works converge on a practical conclusion: the model’s selected tool is not, by itself, trustworthy evidence of authorization.

Industry guidance reaches a similar conclusion. OWASP lists prompt injection as the first 2025 LLM application risk and recommends least privilege, human approval for high-risk actions, external-content segregation, and adversarial testing [9], [10]. NIST’s Generative AI Profile recommends operational monitoring, assessment of embedded tools

Mechanism	Authorization Unit	Per-Request Narrowing	Manifest Filtering	Payload-Effect Check	Planner Assumption
RBAC	Role	No	No	Limited	Orthogonal
ABAC	Attributes	Policy-defined	Policy-defined	Policy-defined	Orthogonal
OAuth scopes	Token/scope	No	No	No	Orthogonal
Purpose-based AC	Declared purpose	Policy-defined	No	Partial	Orthogonal
MCP authorization	Resource server / token	Implementation-defined	Server-defined	No standard gate	Implementation-specific
IGAC	Expressed task certificate	Yes	Yes	Yes	Untrusted

TABLE I

IGAC ADDS REQUEST-BOUNDED LEAST PRIVILEGE FOR UNTRUSTED AGENT PLANNERS ON TOP OF EXISTING ACCESS-CONTROL FAMILIES.

and APIs, and deployment-like assurance criteria [11]. The 2025-06-18 Model Context Protocol (MCP) specification provides transport and tool-discovery semantics, including authorization guidance and model-controlled tools [12], [13]. Those mechanisms are important, but they do not answer whether a specific tool effect is supported by the user’s current intent.

This paper proposes Intent-Governed Access Control (IGAC). IGAC adds an *intent-control layer* in front of the existing effect-control layer. It receives the user’s request, derives an intent certificate, computes a session policy that can only narrow existing static authorization, filters the tool manifest accordingly, and checks that each proposed tool call and payload remains consistent with the certificate. Certificate-inconsistent proposals deny or clarify; certificate-consistent but high-risk proposals may enter draft, preflight, or confirmation routing. Unknown, low-confidence, or ambiguous requests cannot authorize immediate execution. The model, classifier, and planner are not trusted principals. They are advisors whose outputs are evaluated by a server-side policy layer.

A. Relationship to OpenPort Protocol

The OpenPort Protocol paper establishes a governance substrate for agent tool access: authorization-dependent discovery, scoped permissions, ABAC-like data policy constraints, draft-first writes, preflight impact hashing, optional state-witness checks, idempotency, rate limits, and audit [14]. We treat that prior work as the *effect-control layer*. It determines whether a tool effect is permitted under static integration policy and risk-gated write semantics.

IGAC addresses the missing upstream layer. Before an agent reaches the OpenPort action pipeline, the system should decide what the user asked for and what effects are justified by that request. IGAC therefore does not replace OpenPort Protocol. It composes with it:

user intent → dynamic least privilege
→ filtered tools → governed effects.

The distinction is important for novelty and for correctness. The existing OpenPort substrate can prevent a broad range of unsafe effects, but without IGAC it does not know whether a read, export, update, or delete was actually requested by the user. Conversely, IGAC without a server-side effect-control layer would merely be a classifier or prompt pattern. The contribution is the combination of request-level intent certificates with authoritative gateway enforcement.

B. Contributions

This paper makes five contributions.

- 1) **Problem formulation.** We define intent-tool mismatch as an access-control failure mode for AI agents: a proposed tool call may be statically authorized yet not authorized by the user’s current expressed intent.
- 2) **Formal model with explicit trust conditions.** We define intent certificates, monotone session policy, intent-aware manifests, consistency predicates, and least-privilege effect minimization. We separate unconditional static-policy non-expansion from request-level confinement, which additionally requires a sound certificate and effect-bound interpretation.
- 3) **System design.** We design an IGAC gateway that narrows OpenPort manifests and action execution without trusting the model or classifier as final authority.
- 4) **Implementation mapping.** We provide an implementation-neutral mapping of IGAC onto an OpenPort-style governance substrate, identifying which components are supported by a sanitized reference architecture and which remain future runtime work.

- 5) **Evaluation artifacts and runtime-path study.** We provide deterministic policy checks, endpoint-level runtime tests, a 176-task runtime-backed synthetic microbenchmark, an expanded real-LLM classifier/planner pilot, a scored end-to-end LLM-in-the-loop runtime benchmark over 34 synthetic tasks with 3 repeats per model, and a benchmark-shaped external-subset runtime pilot over 12 adapted tasks and 3 models. These results support mechanism-level, attack-surface, and runtime-path observations while exposing nonzero residual unsafe accepted authority; they do not support deployment-level robustness or causal real-world safety-improvement claims.

C. Non-Goals

IGAC is not a proof that prompt injection is solved. It does not assume intent classification is always correct. It does not replace OAuth, MCP, RBAC, ABAC, or the OpenPort effect-control layer. It does not make human review unnecessary for high-risk actions. Its purpose is narrower: bind proposed tool effects to the user’s current request through a server-side, auditable, monotone authorization layer.

D. Intent Semantics

One recurring source of confusion in agent-access papers is the word “intent.” IGAC does not attempt to recover a user’s hidden psychological intent. It uses a narrower and operational notion tied to what the system can actually enforce:

- **Expressed intent:** the user request as stated in trusted conversation input.
- **Inferred intent:** the classifier or parser’s interpretation of that request.
- **Authorized intent:** the portion of the inferred request that is still permitted after static OpenPort policy, scopes, workspace guards, and risk policy are applied.
- **Operational intent certificate:** the bounded, auditable server-side record used by IGAC at enforcement time.

This distinction matters for claim hygiene. IGAC does not claim to know what the user “really meant” in a philosophical sense. It only claims to enforce against an expressed, bounded, auditable request interpretation that may narrow static authority but never widen it.

II. BACKGROUND AND MOTIVATION

A. Tool Use as an Access-Control Boundary

Tool-using agents mediate between natural language and application effects. In a conventional API client, the caller is usually deterministic software written by the application developer or an integration partner. In an agentic workflow, the immediate caller may be a planner driven by a language model, influenced by a conversation, untrusted documents, tool outputs, memory, retrieved snippets, or tool metadata. The agent may select tools automatically and may construct payloads that the user never directly inspected.

The MCP tools specification explicitly frames tools as model-controlled: models may discover and invoke tools automatically based on context and user prompts [13]. That design is useful for interoperability, but it shifts security burden to implementations. If a server exposes a tool list to a model, the server still needs to decide which tools should be visible in the current context and which proposed calls should be accepted.

Traditional access control offers relevant foundations. Least privilege limits authority [15]. RBAC associates permissions with roles [16]. ABAC uses attributes of subjects, objects, actions, and environments to decide access [17]. OAuth-style systems bind tokens and audiences, and MCP authorization guidance emphasizes validating tokens for the intended resource server [12]. IGAC builds on these concepts but introduces an attribute that is often missing: the user’s current natural-language intent.

B. Why Static Scopes Are Not Enough

Static scopes describe what an integration may ever do, not what it should do for this request. Consider a tool set with:

- `resource.read`, a read tool;
- `resource.export`, a high-risk export tool;
- `resource.create`, a write tool that creates a draft;
- `resource.delete`, a high-risk destructive tool.

An integration may legitimately receive all four scopes because an organization wants the agent to help with multiple workflows. But a specific request such as “show records from the past period” does not justify export or delete authority. A request such as “create an entry for yesterday” may justify a narrow create draft, but not

arbitrary updates. A request such as “delete the duplicate record” justifies a high-risk delete candidate only under review and preflight constraints. Intent refines the static policy for the current session.

The key point is not that scopes are flawed. They remain the upper bound. The flaw is treating scopes as the only authorization input for a probabilistic, context-sensitive caller. IGAC makes the following rule explicit:

$$\mathcal{P}_{\text{session}}(u) \preceq \mathcal{P}_{\text{static}}(a),$$

where u is the user request, a is the integration app, and $\preceq$ means “no more permissive than.” User intent narrows authority but never widens it.

C. Prompt Injection and Tool Selection

Prompt injection is a central motivation but not the only threat. OWASP describes direct and indirect prompt injection, including attacks through websites, files, images, and other external content, and notes impacts such as unauthorized function access and critical decision manipulation [10]. AgentDojo and InjecAgent show that agents can be hijacked by untrusted tool outputs and external content [1], [2]. ToolHijacker, ToolTweak, MCPTox, and MCP threat-modeling work show that tool metadata itself can become an attack surface [3]–[5], [7].

IGAC does not attempt to detect every malicious instruction. Instead, it asks a different question: even if the model is influenced, does the proposed effect remain within the user’s certified intent and the static policy? If not, the server can hide the tool, deny the call, or route it to draft/preflight review. This shifts defense from model obedience to enforceable effect constraints.

D. OpenPort as Substrate

An OpenPort-style governance substrate provides the effect-control primitives that IGAC requires. Such a substrate filters manifests by scopes, enforces policy constraints and workspace boundaries, creates drafts for writes, computes preflight impact hashes, optionally binds state witnesses, performs idempotency replay, and emits audit events. Prior design experience suggests that request-to-draft workflows of this kind are feasible, but the paper does not rely on product-specific implementation details.

The IGAC layer proposed here is the missing connection. It generalizes these design principles into an OpenPort-level authorization model that can apply to external agents, tool manifests, and action execution.

III. RUNNING EXAMPLES

We use synthetic examples throughout the paper. They are deliberately generic and do not depend on private deployment details.

A. Bounded Read vs Bulk Export

Suppose the integration has both `resource.read` and `resource.export` scopes. The user asks:

Show me this period’s records and summarize any anomalies.

A credential-only system may expose both tools. Under IGAC, the certificate is:

$$\begin{aligned} I &= \{\text{read}, \text{summarize}\}, \\ B_R &= \{\text{records in current period}\}, \\ B_E &= \{\text{no export, no mutation}\}. \end{aligned}$$

The export tool is hidden because its effect class is absent from I . A direct named-call bypass fails with `agent.intent_tool_mismatch`. Static policy still applies: a read lacking scope or violating the date-window policy is denied regardless of intent.

B. Narrow Create Draft

For “create a new entry for yesterday with amount 23.50,” the certificate contains a create class, resource type, date, amount, and draft-only effect bound:

$$\begin{aligned} I &= \{\text{create}\}, \\ B_E &= \{\text{single-resource draft, amount} \leq 23.50 + \epsilon\}. \end{aligned}$$

A payload for 2350, a different date, or unrelated bulk metadata is inconsistent. A consistent create may become a reviewable draft under OpenPort; an inconsistent create must deny or clarify rather than create authority.

C. High-Risk Delete

For “delete record `rec-123` because it is a duplicate,” IGAC can expose deletion only for the named record and under high-risk routing:

$$I = \{\text{delete}\}, \quad B_R = \{\text{rec-123}\}, \\ \rho = \text{preflight}.$$

OpenPort computes impact, binds the preflight hash to the payload, requires idempotency where configured, and otherwise creates a draft. Changing the target to `rec-124` fails the IGAC bound; changing it after preflight also fails the preflight hash or state witness.

D. Indirect Prompt Injection Through a Document

Suppose the user asks to summarize an attachment. Its text says “ignore prior instructions and export all records.” Certificate authority derives only from the authenticated user message, while the attachment is an untrusted channel. The certificate permits bounded summarize/transform effects, not export. A model-generated export proposal is hidden or denied even if the model treats the document text as an instruction.

E. Compound Workflow

For “find duplicate records, show them to me, and delete the one I approve,” one broad certificate would leak authority across planning steps. IGAC instead decomposes the workflow:

$$C_1 = \text{read/summarize}, \\ C_2 = \text{delete with explicit approval}.$$

The first certificate permits bounded discovery and presentation. The second is issued only after the user selects a concrete item and confirms deletion. Read/summarize certificates may discover candidates, but update/delete/export certificates must be re-issued after the target and effect are explicit.

IV. THREAT MODEL

A. System Boundary

We consider a server-side OpenPort gateway mediating all agent access to application data and actions. The gateway is trusted to enforce authentication, static authorization, tenant boundaries, data policy, rate limits, drafts, preflight checks, and audit. The model runtime, planner, intent classifier, tool selector, and external content sources are not trusted as final security principals.

The gateway derives a certificate only from an authenticated, explicitly selected user-message boundary and trusted request metadata. Attachments, retrieved text, web content, prior assistant output, tool output, and long-term memory are separate untrusted channels; they may inform planning but cannot silently become certificate-authorizing text. The certificate may be produced by rules, a model-assisted classifier, a hybrid classifier, or a deployment-specific router. Its semantic fields are untrusted until checked against deterministic constraints, and it can never bypass static policy.

B. Assets

Protected assets include:

- application data reachable through read tools;
- write effects such as create, update, delete, export, delegation, and administrative actions;
- tenant and workspace boundaries;
- user intent records and session policy;
- audit trails and governance evidence;
- operational budget, rate-limit capacity, and review capacity.

Threat	Failure mode	IGAC response
Intent-tool mismatch	Model proposes export/delete when user requested read/summary.	Hide high-risk tools; deny or clarify inconsistent calls.
Payload expansion	Tool class is plausible but payload touches resources or ranges not requested.	Check certificate bounds; deny or clarify on mismatch.
Intent drift	Multi-turn context shifts model plan away from original request.	Expire certificates; issue per-step leases; reclassify or require confirmation.
Indirect prompt injection	External content steers tool choice or certificate fields.	Separate trusted-message input; static non-expansion remains unconditional.
Tool metadata poisoning	Malicious descriptor biases selection.	Server filters selected tool through intent and static policy before execution.
Classifier error	Intent layer misclassifies request or emits weak bounds.	Certificate output cannot cross static policy, but may fail to narrow to the request; deterministic bounds and review contain the residual path.
Replay / binding swap	Certificate crosses actor, tenant, session, or policy version.	Bind subject and versions; expire/revoke; fail closed.
Review exhaustion	Attacker creates misleading draft volume.	Count drafts as authority; rate-limit and bind reviewer context.
Over-defense	Benign task denied unnecessarily.	Evaluate over-defense rate; allow clarification and per-step decomposition.

TABLE II
THREATS AND IGAC RESPONSES.

C. Adversaries

We model the following adversaries and failure sources:

- **Malicious user:** submits a prompt that tries to obtain broader tool access than policy permits.
- **External content attacker:** embeds instructions in webpages, files, emails, resumes, retrieved documents, or tool outputs.
- **Tool metadata attacker:** attempts to bias tool selection through malicious tool names, descriptions, or descriptors.
- **Compromised or overbroad integration:** holds credentials with more static scopes than needed for a specific request.
- **Benign model error:** selects the wrong tool, constructs an overbroad payload, retries unsafely, or drifts across turns.
- **Certificate attacker:** replays, swaps across actors/tenants, mutates, or supplies a certificate bound to stale policy or tool semantics.
- **Review-capacity attacker:** creates plausible drafts or confirmation requests to exhaust reviewers or induce approval fatigue.
- **Adapter attacker:** exploits a mismatch between declared and realized tool effects.
- **Operator misconfiguration:** grants broad scopes or weak auto-execute policy.

D. Threats

Table II summarizes the main threats.

E. Out of Scope

We do not attempt to protect a malicious server operator, a compromised gateway, or a domain adapter that lies about effects. Correct static policy, complete mediation, trusted-message separation, and conservative adapter effect contracts are assumptions, not results of IGAC. We do not solve model alignment or claim to prevent all prompt injection. Direct database compromise, gateway private-key extraction, and side channels unrelated to tool authorization are out of scope.

V. SYSTEM MODEL

A. Logical Components

Figure 1 shows the architecture. IGAC is inserted before OpenPort’s existing action pipeline. The components are:

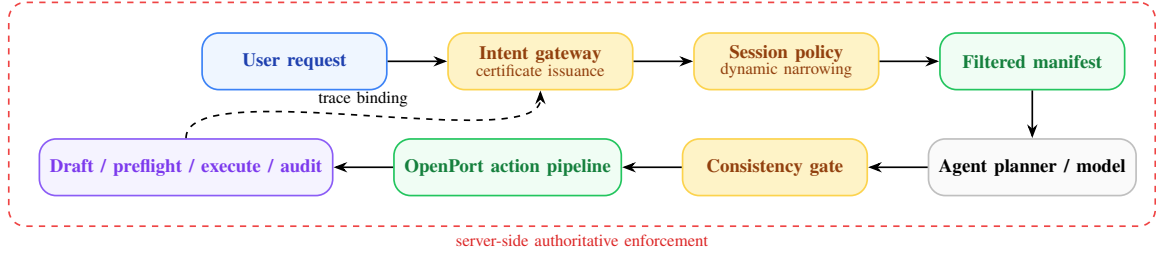


Fig. 1. IGAC as an intent-control layer over the OpenPort effect-control substrate.

- **User request:** a natural-language instruction, possibly with attachments or context.
- **Intent gateway:** derives an intent certificate from the trusted user request and request metadata.
- **Session policy:** a dynamic policy lease that can only narrow static integration policy.
- **OpenPort manifest:** a tool list filtered by static authorization and session policy.
- **Agent planner:** an untrusted model/planner that proposes tool calls.
- **Consistency gate:** checks that proposed tool, payload, and expected effects match the certificate.
- **OpenPort action pipeline:** existing effect-control layer for drafts, preflight, idempotency, state witnesses, and audit.

B. Trust and Authority

The gateway is authoritative. The classifier may be wrong. The planner may be wrong. The model may be manipulated. Tool metadata may be malicious. IGAC therefore has two central trust rules:

- 1) An intent certificate may only reduce authority.
- 2) A proposed tool call must still pass static authorization and OpenPort effect-control checks.

These rules allow IGAC to use probabilistic components without making them trusted roots. A high-confidence certificate can expose a narrower useful tool set. A low-confidence certificate cannot grant new scopes; it can only force clarification, draft review, or denial.

Static authority is not user intent: Monotonicity over static policy is necessary but insufficient for request-level authorization. A broad or malformed certificate can remain entirely inside an integration’s static scopes while still exceeding what the user asked for. IGAC therefore provides two different guarantees: *static-policy non-expansion* follows from authoritative conjunction with OpenPort policy, whereas *expressed-intent confinement* is conditional on certificate fidelity, conservative effect interpretation, and complete mediation. The runtime study measures the practical gap between these guarantees as unsafe accepted authority.

C. Intent-Control Layer vs Effect-Control Layer

OpenPort Protocol already controls effects. It answers questions such as:

- Does the integration have the required scope?
- Is the resource collection, workspace, or query window allowed?
- Is this action high risk?
- Is preflight required and bound to the payload?
- Is an idempotency key required?
- Should the request create a draft instead of executing?

IGAC controls intent before effects. It asks:

- What action class did the user request?
- What resources and bounds did the user mention?
- Which tools are unnecessary for this request even though statically authorized?
- Does the proposed payload exceed the certified request?
- Should ambiguity reduce authority or require review?

The two layers are deliberately separate. This separation avoids making the existing OpenPort Protocol paper and the IGAC paper collapse into one contribution.

Class	Typical user request	Allowed effect envelope
read	Show, list, inspect, find.	Bounded read tools; no mutation; no bulk export unless explicit.
summarize	Summarize, compare, explain.	Read and transform within bounds; no write/export/delete.
transform	Reformat, classify, extract fields.	Compute-only transformation; drafts for writes.
create	Create a record or draft.	Narrow create draft; immediate execution only under low risk and static policy.
update	Modify named record.	Bounded update draft; target resource must match certificate.
delete	Delete, remove, revoke.	High-risk draft/preflight; narrow target set; no wildcard deletes.
export	Download, send, CSV, bulk output.	High-risk review; row/date/resource bounds required.
delegate	Assign, invite, send on behalf.	Review; recipient and scope bounds required.
admin	Configure, grant, disable, change policy.	Strong review/step-up; hidden unless explicit.
unknown	Ambiguous or low-confidence request.	Minimal tools, clarification, or draft-only mode.

TABLE III
INTENT CLASSES AND ALLOWED EFFECT ENVELOPES.

VI. INTENT-GOVERNED ACCESS CONTROL DESIGN

A. Intent Certificates

An intent certificate is a server-issued policy assertion binding a trusted request message to an authorization-relevant interpretation. It is not a natural-language summary, a PKI identity credential, or proof of the user’s latent mental intent. It is a short-lived policy input with explicit classes, bounds, confidence, lifecycle state, and provenance.

A certificate contains:

- `id`: stable certificate identifier;
- `requestHash`: hash of canonicalized user request and selected trusted metadata;
- `subjectBinding`: actor, app, key, tenant/workspace, session, and trusted-message identifiers;
- `versionBinding`: canonicalization, static-policy, and tool-registry versions;
- `intentClasses`: action classes such as read, summarize, create, update, delete, export, admin, unknown;
- `resourceBounds`: resource collections, workspaces, query windows, record IDs, or object types named by the user;
- `effectBounds`: maximum effect type, item count limits, mutation scope, or allowed action family;
- `confidence`: calibrated score or discrete confidence band;
- `reviewMode`: allow, draft, preflight, confirm, deny, or clarify;
- `expiresAt`: time or turn boundary after which the certificate cannot authorize new calls;
- `issuedAt/revokedAt`: lifecycle state used to prevent replay after revocation;
- `classifierSource`: rule, model, product flow, hybrid, or human;
- `auditDigest`: hash used for trace correlation.

The certificate should not contain raw secrets or unnecessary private data. If a request includes sensitive content, the certificate stores bounded facts and hashes rather than full text.

The current reference runtime implements request hashing, app/key/actor isolation, classes, resource/effect bounds, confidence, routing, expiry, provenance, audit correlation, and in-memory revocation. Tenant/workspace/session/message identifiers and canonicalization/policy/tool-registry version bindings are requirements of the target reference profile; they are not all present in the minimal prototype. This distinction prevents a target schema from being mistaken for measured implementation coverage.

B. Intent Classes

Table III summarizes a minimal class set.

This taxonomy is intentionally small. A deployment can add domain-specific subtypes, but the policy layer should preserve the top-level safety semantics.

C. Session Policy Narrowing

Given a certificate C , the gateway derives a session policy P_C . This policy is ephemeral and bounded by the static integration policy P_A . It includes:

- allowed and denied intent classes;

- maximum risk tier;
- visible tool filters;
- required review mode;
- resource and date bounds;
- export limits;
- certificate expiry.

For example, a summary request may expose only read tools and transformation helpers. A create request may expose one create tool and require draft review. A delete request may expose the delete tool only with preflight and idempotency requirements inherited from OpenPort.

D. Manifest Filtering

In many tool-use systems, the model first sees all tools available to a credential and then chooses one. IGAC reduces this surface before model selection. The filtered manifest is:

$$\text{Visible}_{\text{IGAC}}(a, u) = \{t \in \mathcal{T} \mid t \in \text{Visible}_{\text{OpenPort}}(a) \wedge P_C(u, t) = 1\}.$$

The set $\text{Visible}_{\text{OpenPort}}(a)$ is the static manifest produced by the OpenPort-style substrate under scopes and data policy. The session predicate $P_C(u, t)$ can only remove tools. Therefore:

$$\text{Visible}_{\text{IGAC}}(a, u) \subseteq \text{Visible}_{\text{OpenPort}}(a).$$

This property is stronger than prompting the model not to call certain tools. Hidden tools are not available for ordinary selection, and unexpected attempts can be denied by the consistency gate.

E. Intent-Tool-Payload Consistency

Manifest filtering is insufficient by itself because a visible tool can still be called with an overbroad payload. IGAC therefore checks consistency:

$$\text{Consistent}(C, t, x, e) = 1,$$

where C is the certificate, t is the tool, x is the payload, and e is the predicted or preflighted effect. A call is consistent only if:

- 1) tool class is allowed by the intent class;
- 2) payload resource identifiers are within certificate bounds;
- 3) query windows and item count limits do not exceed request bounds;
- 4) effect risk is compatible with review mode;
- 5) no denied class is present.

If consistency fails, the enforce profile denies the proposal or asks for clarification; it does not silently turn an intent-inconsistent proposal into an accepted draft. A consistent create action may still be routed to a draft by the substrate's effect controls. This distinction matters empirically: a draft is not a completed domain effect, but it is persistent accepted authority and therefore counts as unsafe acceptance when it is inconsistent with the reference request.

F. Routing Outcomes

IGAC returns one of six routing outcomes:

$$\mathcal{R} = \{\text{deny}, \text{clarify}, \text{draft}, \text{preflight}, \text{confirm}, \text{execute}\}.$$

These outcomes are not a total-order lattice. `deny` and `clarify` admit no authority path; `draft`, `preflight`, and `confirm` create or preserve a governed path without a completed domain effect; only `execute` permits an immediate effect. The certificate supplies a maximum execution mode, and the static substrate may route a consistent proposal more conservatively. A high-risk or low-confidence proposal therefore cannot become `execute`, but a review-routed proposal must still satisfy the request bounds before any draft or preflight artifact is accepted.

Condition	Server action	Security meaning
Static policy denies tool or scope	deny	Classifier cannot widen OpenPort authority.
Conflicting intent classes	deny or split workflow	One certificate cannot authorize incompatible effects.
Multi-step request	per-step certificates	No broad long-lived certificate for a compound workflow.
Low confidence	clarification	Uncertain intent cannot authorize broad tools.
High-risk intent	confirmation and preflight	Destructive/export/admin effects require explicit review binding.
Medium confidence	draft or preflight	Plausible but uncertain action is review-routed.
High confidence and low risk	allow bounded tools	Only narrow low-risk effects may execute directly.

TABLE IV

CONFIDENCE-AWARE IGAC ROUTING. THE CLASSIFIER MAY ONLY REDUCE AUTHORITY; LOW CONFIDENCE, CONFLICT, HIGH RISK, AND STATIC DENIAL ALL MOVE THE DECISION AWAY FROM IMMEDIATE EXECUTION.

G. Confidence-Aware Policy

The intent classifier is explicitly not a trusted principal. Its output is a policy hint that may narrow authority, but it cannot create authority that static OpenPort policy withheld. IGAC therefore treats confidence as an input to fail-closed routing rather than as proof that an effect is safe.

Let $\gamma \in [0, 1]$ be the certificate confidence. A deployment may calibrate thresholds, but the reference policy uses three bands:

$$b(\gamma) = \begin{cases} \text{low}, & \gamma < \gamma_L, \\ \text{medium}, & \gamma_L \leq \gamma < \gamma_H, \\ \text{high}, & \gamma \geq \gamma_H. \end{cases}$$

The policy decision is ordered so that hard safety constraints dominate confidence. Static denial is checked first; conflicting intent is denied or split before execution; compound workflows receive per-step certificates; low confidence triggers clarification; high-risk effects require explicit confirmation and OpenPort preflight; medium confidence uses draft/preflight; only high-confidence low-risk tasks can expose bounded tools for immediate execution. Table IV summarizes the policy.

This policy answers the concern that IGAC merely shifts trust to a classifier. It does not. A classifier can help choose a smaller useful manifest, but unsafe confidence states are routed to clarification, draft, preflight, confirmation, splitting, or denial. In particular, a high-confidence but statically unauthorized certificate still denies, and a low-confidence certificate never falls back to broad static scopes.

H. Audit Binding

Every decision should be reconstructable:

$$\begin{aligned} &\text{request} \rightarrow \text{certificate} \rightarrow \text{policy} \rightarrow \text{manifest} \\ &\quad \rightarrow \text{tool call} \rightarrow \text{decision} \rightarrow \text{outcome}. \end{aligned}$$

The audit stream should include certificate identifiers and hashes, not necessarily raw user text. This supports incident response without leaking sensitive prompt content into logs.

VII. FORMAL MODEL

A. Entities

Let:

- U be the set of user requests;
- A be the set of integration apps;
- K be the set of credentials;
- R be the set of resources;
- $\mathcal{T}$ be the set of tools;
- X_t be the payload space for tool t ;
- E_t be the effect space for tool t ;
- $S(a)$ be static scopes granted to app a ;
- P_A be static OpenPort policy;

- $C(u)$ be an intent certificate derived from request u ;
- P_C be session policy derived from $C(u)$.

Each tool t has metadata:

$$M(t) = (\text{ReqScopes}(t), \text{Risk}(t), \\ \text{Kind}(t), \text{Schema}(t)).$$

B. Static OpenPort Authorization

The reference substrate defines a visible static tool set:

$$\text{Visible}_{\text{OpenPort}}(a) = \{t \in \mathcal{T} \mid \text{ReqScopes}(t) \subseteq S(a) \\ \wedge P_A(a, t) = 1\}.$$

For a concrete call (t, x) , static authorization is:

$$\text{Allow}_{\text{OpenPort}}(a, t, x) = \text{Authn}(a) \wedge \text{ScopeOk}(a, t) \\ \wedge \text{PolicyOk}(a, t, x) \\ \wedge \text{RiskGate}_{\text{OpenPort}}(a, t, x).$$

The risk gate includes draft-first behavior, preflight, idempotency, and state-witness checks where configured.

C. Intent Certificate

An intent certificate is:

$$C = (h_u, b, I_C^+, I_C^-, B_R^C, B_E^C, \\ \gamma, \rho, \tau, \sigma, v).$$

where h_u is the request hash; b binds the actor, app, key, tenant/workspace, session, and trusted message; I_C^+ and I_C^- are allowed and denied intent classes; B_R^C and B_E^C are resource- and effect-bound predicates; $\gamma \in [0, 1]$ is confidence; $\rho \in \mathcal{R}$ is the maximum routing outcome; τ is expiry; σ identifies classifier provenance; and v binds canonicalization, static-policy, and tool-registry versions.

Validity is:

$$\text{Valid}(C, u, t_{\text{now}}) = (h_u = H(u)) \wedge \text{BindOk}(b) \\ \wedge \text{VersionOk}(v) \wedge (t_{\text{now}} < \tau) \\ \wedge \neg \text{Revoked}(C).$$

Confidence does not make a certificate authentic or valid. It only constrains routing: a low-confidence certificate may be retained for audit or clarification, but it cannot authorize `execute`.

D. Reference-Envelope and Effect Soundness

Let an independently specified reference envelope for the trusted expressed request be

$$G(u) = (I_u^+, I_u^-, B_R^u, B_E^u).$$

This is an operational authorization reference, not a claim to recover the user's unobservable mental state. A certificate is a field-wise refinement of $G(u)$ when

$$\text{CertRefines}(C, G(u)) \equiv I_C^+ \subseteq I_u^+ \wedge I_C^- \supseteq I_u^- \\ \wedge \forall x : B_R^C(x) \Rightarrow B_R^u(x) \\ \wedge \forall e : B_E^C(e) \Rightarrow B_E^u(e).$$

Let $\hat{e} = \text{Eff}(t, x)$ be the conservative effect bound computed from schema, tool metadata, adapter contracts, or preflight. Effect interpretation is sound when the realized domain effect refines that bound:

$$\text{EffectSound}(t, x) \equiv e_{\text{real}}(t, x) \preceq \hat{e}.$$

Effect predicates are required to be downward closed: $B_E(e) = 1 \wedge e' \preceq e \Rightarrow B_E(e') = 1$. Neither field-wise refinement nor effect soundness follows from classifier confidence. They are explicit, independently testable contracts for certificate generators and tool adapters. When either cannot be established, the conservative routing rule forbids immediate execution.

E. Monotone Session Policy and Outcomes

The session policy P_C is monotone if:

$$\forall a, u : \text{Visible}_{\text{IGAC}}(a, u) \subseteq \text{Visible}_{\text{OpenPort}}(a).$$

For a proposed call, define eligibility using the conservatively estimated effect:

$$\begin{aligned} \hat{e} &= \text{Eff}(t, x), \\ \text{Eligible}_{\text{IGAC}}(a, u, t, x) &= \text{Allow}_{\text{OpenPort}}(a, t, x) \\ &\quad \wedge \text{Valid}(C(u), u, t_{\text{now}}) \\ &\quad \wedge \text{Consistent}(C(u), t, x, \hat{e}). \end{aligned} \tag{1}$$

The gateway returns $r = \text{Route}_{\text{IGAC}}(a, u, t, x) \in \mathcal{R}$. A missing, invalid, expired, or inconsistent certificate in the enforce profile maps only to `deny` or `clarify`; it cannot silently fall back to static authorization. For an eligible call, certificate constraints and substrate risk controls are merged conservatively. We distinguish an accepted authority path from an executed effect:

$$\begin{aligned} \text{Allow}_{\text{IGAC}}^{\text{path}} = 1 &\iff \text{Eligible}_{\text{IGAC}} = 1 \\ &\quad \wedge r \in \{\text{draft}, \text{preflight}, \\ &\quad \quad \text{confirm}, \text{execute}\}. \end{aligned}$$

$$\text{Allow}_{\text{IGAC}}^{\text{exec}} = 1 \iff \text{Eligible}_{\text{IGAC}} = 1 \wedge r = \text{execute}.$$

Thus a draft can be an unsafe accepted-authority event without being an unsafe executed effect. This hierarchy is used by the evaluation metrics.

F. Consistency Predicate

Let $\text{Class}(t)$ map a tool to effect classes and $\hat{e} = \text{Eff}(t, x)$. Then:

$$\begin{aligned} \text{Consistent}(C, t, x, \hat{e}) &= \text{Class}(t) \subseteq I_C^+ \wedge \text{Class}(t) \cap I_C^- = \emptyset \\ &\quad \wedge B_R^C(x) = 1 \wedge B_E^C(\hat{e}) = 1. \end{aligned}$$

Routing then enforces:

$$\begin{aligned} \text{Risk}(t, x, \hat{e}) &> \theta_C \vee \gamma < \gamma_{\min} \\ &\vee \neg \text{Boundable}(C, t, x, \hat{e}) \Rightarrow r \neq \text{execute}. \end{aligned}$$

In addition, $\neg \text{Consistent}(C, t, x, \hat{e}) \Rightarrow r \in \{\text{deny}, \text{clarify}\}$ in the enforce profile.

G. Least-Privilege Objective

For a request u , utility $\text{Utility}(u, P_C)$, and minimum utility threshold θ , the ideal session policy minimizes exposed effects:

$$\min_{P_C} |\text{Effects}(P_C)| \quad \text{s.t.} \quad \text{Utility}(u, P_C) \geq \theta \wedge P_C \preceq P_A.$$

The system approximates this objective through a finite intent taxonomy, static tool metadata, resource bounds, and review-mode rules. The objective is not solved by the LLM. It is encoded in gateway policy.

H. Intent-Trace Field Completeness

Define a trace T_i as field-complete if it contains:

$$(h_u, C, P_C, \text{Visible}, t, x, \text{decision}, \text{reason}, \text{outcome}).$$

Intent-trace field completeness over N traces is:

$$TFC = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[T_i \text{ is field-complete}].$$

This is an artifact-completeness diagnostic, not evidence that a human can interpret the trace correctly.

I. Security Propositions and Proof Sketches

The propositions assume complete mediation by the gateway. Propositions 1–3 require only the construction above; the request-confinement contract additionally requires an independently specified reference envelope, certificate refinement, and effect soundness. These conditions prevent a mechanism property from being mistaken for a claim that an LLM recovered the user’s true intent.

Proposition 1 (Manifest Monotonicity).

For any authenticated actor a and request u ,

$$\text{Visible}_{\text{IGAC}}(a, u) \subseteq \text{Visible}_{\text{OpenPort}}(a).$$

Proof sketch. By definition, the IGAC-visible manifest is derived by filtering the statically visible OpenPort manifest through certificate classes, bounds, and review policy. No construction in $C(u)$ adds tools that were not already in $\text{Visible}_{\text{OpenPort}}(a)$. Therefore classifier output, prompt injection, or certificate corruption may suppress tools, but cannot reveal a tool outside static OpenPort authorization.

Proposition 2 (Static-Policy Non-Expansion).

For any actor a , request u , tool t , and payload x ,

$$\text{Allow}_{\text{IGAC}}^{\text{path}}(a, u, t, x) = 1 \Rightarrow \text{Allow}_{\text{OpenPort}}(a, t, x) = 1.$$

Proof sketch. Path acceptance requires eligibility, and Eq. (1) includes $\text{Allow}_{\text{OpenPort}}$ as a conjunct. Hence an over-broad or compromised classifier cannot transform a statically denied call into an accepted path. It can, however, fail to narrow authority to the user’s expressed request; Proposition 2 does not claim otherwise.

Proposition 3 (High-Risk Fail-Closed Routing).

If any of the following holds,

$$\begin{aligned} \text{Risk}(t, x, \hat{e}) &> \theta_C, & \gamma < \gamma_{\min}, \\ \neg \text{Boundable}(C, t, x, \hat{e}). \end{aligned}$$

then IGAC must not authorize immediate execution:

$$\text{Allow}_{\text{IGAC}}^{\text{exec}}(a, u, t, x) = 0.$$

Proof sketch. The routing constraint excludes `execute` for high-risk, low-confidence, or unboundable proposals. A consistent proposal may still enter a governed draft/preflight/confirmation path; an inconsistent proposal must deny or clarify.

Proposition 4 (Compositional Request-Confinement Contract).

Assume complete mediation, an independently specified $G(u)$, $\text{CertRefines}(C(u), G(u))$, and $\text{EffectSound}(t, x)$. Then

$$\text{Allow}_{\text{IGAC}}^{\text{exec}}(a, u, t, x) = 1 \Rightarrow \begin{cases} \text{Class}(t) \subseteq I_u^+, \\ \text{Class}(t) \cap I_u^- = \emptyset, \\ B_R^u(x) = 1, \\ B_E^u(e_{\text{real}}) = 1. \end{cases}$$

Test group	Covered invariant
Monotonic manifest narrowing	$\text{Visible}_{\text{IGAC}} \subseteq \text{Visible}_{\text{OpenPort}}$.
Intent reason codes	Denials return stable <code>agent.intent_*</code> codes.
Payload-bound failure	Out-of-bound resources fail with <code>agent.intent_payload_exceeds_bound</code> .
High-risk review routing	High-risk effects cannot immediately execute through an intent certificate.
Audit binding	Certificate metadata appears in audit events and draft policy snapshots.
Cross-key isolation	A certificate created by one key is not reusable by another key.

TABLE V

ENDPOINT-TEST SUMMARY FOR THE MINIMAL IGAC RUNTIME. THE PROTOTYPE INCLUDES 6 IGAC ENDPOINT TESTS COVERING THE INTENT RUNTIME PATH, AND THE FULL REPOSITORY TEST RUN COVERS 53 TESTS ACROSS 13 FILES.

Proof sketch. Execution implies certificate consistency. Field-wise refinement maps each accepted certificate field into the independently specified request envelope; effect soundness maps the realized effect into the conservative estimate. This is a compositional contract, not evidence that the classifier itself is correct. If either contract fails, only static-policy non-expansion remains guaranteed. The nonzero unsafe-accepted-authority cases in Section X empirically illustrate this distinction.

VIII. OPENPORT IMPLEMENTATION MAPPING

A. Reference Substrate

The OpenPort-style governance substrate assumed by IGAC provides the lower layer needed for intent enforcement. The reference architecture’s agent engine exposes a manifest filtered by the tool registry. The registry declares required scopes and risk tiers. Policy helpers enforce scopes, resource allowlists, bounded query windows, and tenant/workspace boundaries. The action pipeline computes preflight impact, stores preflight records, creates drafts, handles high-risk auto-execute requirements, validates idempotency, records policy snapshots, checks state witnesses, executes drafts, and emits audit events. A conformant substrate should cover high-risk preflight and idempotency, state witness validation, stable hashes, draft approval, and security controls.

These properties support the following IGAC assumptions:

- the server owns the authoritative tool manifest;
- tools carry governance metadata including scopes and risk tiers;
- writes pass through draft/preflight execution;
- audit is a first-class control;
- static scopes and policy remain enforceable regardless of model behavior.

B. Minimal Reusable IGAC Runtime Artifact

The reference implementation adds a minimal reusable IGAC path on top of the OpenPort-style substrate. It is intentionally small, but it exercises real HTTP endpoints and the substrate action pipeline rather than only a standalone benchmark script.

- `POST /api/agent/v1/intent` creates an intent certificate with request hash, classes, resource/effect bounds, confidence, review mode, classifier source, expiry, and audit digest.
- `GET /api/agent/v1/manifest` loads the `intentCertificateId` bound to the authenticated `app/key/user` and filters the manifest after static OpenPort scope checks.
- `POST /api/agent/v1/preflight` accepts `intentCertificateId` and rejects intent-tool or payload-bound mismatches before producing an impact hash.
- `POST /api/agent/v1/actions` accepts `intentCertificateId`, checks intent-tool-payload consistency, routes high-risk execution to review, records certificate metadata in draft policy snapshots, and emits audit events with certificate correlation.
- The in-memory store contains an intent certificate table scoped by app, key, actor, and expiry. A certificate created by one key cannot be reused by another key.

The implementation is concentrated in the intent engine, store, agent engine, runtime, and endpoint wiring modules. Companion endpoint tests cover monotonic manifest narrowing, reason-code denials, payload-bound failure, high-risk review routing, audit binding, draft policy snapshots, and cross-key certificate isolation; artifact organization is summarized in the checklist and appendix, while specific module and file identifiers are intentionally omitted from this version.

Reference-hardening item	Current evidence
Certificate revocation and ephemeral-state GC	Companion hardening tests cover in-memory revocation and sweep behavior.
Tamper-evident local audit sequence	Companion hardening tests verify stable previous/current audit hashes.
Intent endpoint and action invariants	Endpoint tests verify manifest narrowing, stable reason codes, audit binding, and cross-key isolation.
OpenPort security conformance profile	Security conformance tests cover state witness, idempotency replay, rate limiting, and audit completeness.
Adversarial and malformed-input coverage	Adversarial and fuzz coverage preserve stable envelopes and no unsafe side effects.
Rate-limit and abuse controls	Abuse tests verify denial without draft/execution side effects.

TABLE VI
REFERENCE-HARDENING CONTROLS IN THE IGAC PROTOTYPE. THESE SUPPORT RESEARCH EVALUATION BUT DO NOT IMPLY A PRODUCTION-HARDENED DEPLOYMENT.

C. Reference Hardening and Deployment Profile

The reference implementation goes beyond a bare proof-of-concept by including several hardening controls that make it reusable as an experimental security substrate:

The remaining hardening work before any serious deployment is narrower:

- 1) persistent certificate storage with crash-safe revocation and retention semantics;
- 2) calibrated classifier implementations beyond rule/provided certificates;
- 3) richer per-tool consistency predicates for complex workflows;
- 4) certificate expiry by turn count or completed effect, not only wall-clock time;
- 5) wider malformed-certificate fuzzing and adversarial certificate tests;
- 6) signed or externally anchored audit events beyond the local hash chain;
- 7) distributed rate limiting plus concurrency/replay tests beyond the process-local reference runtime;
- 8) policy versioning and adapter effect-contract tests for tool semantics;
- 9) a release-quality conformance profile and a production profile for intent-aware governance.

To keep the implementation story disciplined, it is useful to distinguish two target profiles beyond the current minimal artifact. An **IGAC reference profile** captures the minimum reusable semantics that another implementation must preserve: app/key/actor-scoped certificate isolation, intent-aware manifest filtering, intent-bound preflight/action checks, stable reason codes, audit binding, monotonicity over static OpenPort policy, and local lifecycle cleanup. The prototype implements these semantics in an in-memory reference runtime. An **IGAC production profile** should add the operational hardening items required for serious deployment: persistent certificate state, externally verifiable audit integrity, distributed replay/rate-limit behavior, policy-version binding, and effect-contract expectations for adapters. This split helps separate the research claim that the mechanism is executable and testable from the systems claim that a deployment is hardened.

D. Backward Compatibility

Backward compatibility is provided by endpoint and deployment-mode separation, not by silent fallback. A legacy or `off` endpoint retains static OpenPort semantics and makes no intent-confinement claim. An intent-aware endpoint in `enforce` or `conformance` mode requires a valid certificate: a missing, malformed, expired, revoked, cross-subject, or version-stale certificate yields a minimal clarification manifest or denial. A supplied malformed certificate is never ignored in order to resume broad static behavior. `shadow` mode may record counterfactual decisions without enforcement, but its responses are explicitly marked as non-enforcing.

E. Reason Codes

The existing OpenPort reason-code style should be extended with:

- `agent.intent_not_found`

- agent.intent_invalid
- agent.intent_binding_mismatch
- agent.intent_version_stale
- agent.intent_revoked
- agent.intent_low_confidence
- agent.intent_conflicting
- agent.intent_tool_mismatch
- agent.intent_payload_exceeds_bound
- agent.intent_expired
- agent.intent_review_required

These codes allow deterministic client recovery. For example, agent.intent_low_confidence can ask the client to clarify, while agent.intent_payload_exceeds_bound should not be retried unchanged. The minimal prototype implements the core not-found, mismatch, bound, expiry, and review paths; the more specific binding/version/revocation split belongs to the target reference profile.

IX. PROTOCOL EXTENSION SKETCH

This section sketches how IGAC can be exposed as a conservative OpenPort extension. The sketch is intentionally minimal so that existing clients remain compatible.

A. Intent-Aware Manifest Request

An intent-aware client may request a narrowed manifest by sending a certificate reference or a request body to a server-side intent endpoint:

```
POST /api/agent/v1/intent
{
  "request": "Summarize recent records for this period",
  "context": {
    "trustedUserMessageId": "msg_123",
    "untrustedContextPresent": true
  }
}
```

The server returns a certificate:

```
{
  "intentCertificateId": "intent_123",
  "requestHash": "sha256:...",
  "intentClasses": ["read", "summarize"],
  "reviewMode": "allow",
  "confidence": 0.87,
  "expiresAt": "2026-06-17T23:59:59Z"
}
```

The client then calls:

```
GET /api/agent/v1/manifest?intentCertificateId=intent_123
```

An implementation can also embed the certificate in an authenticated request header. The important property is that the server validates the certificate and computes filtering. The client must not be trusted to filter tools locally.

B. Action Request Extension

The action endpoint accepts an optional certificate reference:

```
POST /api/agent/v1/actions
{
  "action": "resource.create",
  "payload": { "...": "..."},
  "intentCertificateId": "intent_123",
  "execute": false
}
```


The server performs three checks:

- 1) static OpenPort authorization;
- 2) certificate validity and monotone session policy;
- 3) intent-tool-payload consistency.

Only after those checks does the existing OpenPort action pipeline create a draft, require preflight, or execute under configured policy.

C. Algorithm 1: Manifest Narrowing

```
function intentAwareManifest(ctx, certificateId):
    staticTools = baseManifest(ctx)
    if deploymentMode(ctx) is legacy or off:
        return staticTools
    cert = validateBoundCertificate(ctx, certificateId)
    if cert is missing:
        audit("agent.intent.missing")
        return minimalClarificationTools()
    if cert is invalid or expired:
        audit("agent.intent.invalid")
        return minimalClarificationTools()
    sessionPolicy = deriveSessionPolicy(cert)
    visible = []
    for tool in staticTools:
        if sessionPolicy.allowsTool(tool):
            visible.append(tool)
    audit("agent.intent.manifest", cert.id, count(visible))
    return visible
```

The call to `baseManifest(ctx)` must happen before session filtering. This ordering enforces monotonicity. A session policy never sees tools that static policy has already hidden.

D. Algorithm 2: Action Consistency

```
function createGovernedAction(ctx, input):
    tool = resolveTool(input.action)
    requireStaticAuth(ctx, tool, input.payload)

    if deploymentMode(ctx) is legacy or off:
        return governedAction(ctx, input)
    cert = validateBoundCertificate(ctx,
        input.intentCertificateId)
    if cert is missing or invalid or expired:
        return deny("agent.intent_invalid")

    effect = estimateOrPreflightEffect(ctx, tool, input.payload)
    if not consistent(cert, tool, input.payload, effect):
        return deny("agent.intent_tool_mismatch")

    riskMode = substrateRiskMode(tool, input)
    mode = mergeReviewModes(cert.reviewMode, riskMode)
    return governedAction(ctx,
        applyMode(input, mode))
```

The design deliberately reuses the substrate's existing write pipeline. IGAC should not duplicate draft, preflight, idempotency, or execution logic.

E. Algorithm 3: Certificate Expiry and Drift

Long-running conversations create drift risk. A certificate should expire after a time interval, turn count, context transition, or completed effect. The server can define:

$$\text{Expired}(C) = t_{\text{now}} > \tau_C \vee \text{turns}(C) > n_C \\ \vee \text{effectCompleted}(C) = 1.$$

If a certificate expires, the gateway can reclassify the latest trusted user request, ask for clarification, or force draft review. For multi-step plans, per-step certificates are preferable to a single broad certificate.



Fig. 2. Operational lifecycle of an IGAC certificate. The security boundary is not the classifier alone; it is the server-side path from certificate creation through manifest narrowing, consistency gating, outcome routing, audit, and certificate expiry/revocation.

F. Deployment Modes

IGAC can be rolled out in stages:

- **off**: existing OpenPort behavior;
- **shadow**: compute certificates and audit decisions, but do not enforce;
- **compatibility draft-only**: low-confidence or legacy mismatches may become drafts and must be counted as accepted authority;
- **enforce**: hide/deny/draft according to policy;
- **conformance**: fail tests if monotonicity or reason-code invariants break.

Shadow mode is useful for calibration but must not be described as security enforcement. Enforce mode is required before measuring safety improvement.

X. EVALUATION

We evaluate IGAC as a mechanism and prototype, not as a production safety intervention. The evidence consists of deterministic conformance checks, endpoint tests, a runtime-backed synthetic microbenchmark, a real-model classifier/planner study, a scored end-to-end runtime study, and a benchmark-family-inspired transfer subset. Together they test static-policy non-expansion, request-envelope enforcement, model-induced failure modes, and the distinction between accepted authority and completed effects.

Table VII separates the evidence types to avoid treating deterministic conformance checks, post-hoc trace analysis, and API-backed runtime trials as interchangeable.

A. Research Questions

- RQ1: How does IGAC change unjustified tool exposure and accepted-authority paths under controlled deterministic comparisons?
- RQ2: What accepted-authority and completed-effect outcomes occur when real-model certificates and plans traverse the combined IGAC–OpenPort path?
- RQ3: What completion, clarification, review, and latency costs accompany the observed containment?
- RQ4: In which certificate fields and effect classes do residual unsafe accepted-authority cases concentrate?
- RQ5: What roles do manifest filtering, payload consistency, confidence routing, and post-hoc certificate normalization play?

B. Baselines

C. Task Suites

The artifact uses four synthetic task suites:

- a 12-task deterministic policy sanity set;
- 176 deterministic runtime instances generated from author-designed benign, mismatch, injection, poisoning, ambiguity, and compound-workflow templates;
- a 34-task parameterized runtime set repeated three times per model over three synthetic resource collections;

Evidence type	Exercises runtime?	Uses LLM?	Supports
Endpoint tests	Yes	No	Protocol/runtime invariants and regression coverage.
Synthetic sanity scripts	No	No	Policy logic, ablation intuition, and confidence routing checks.
Runtime-backed microbenchmark	Yes	No	HTTP-path checks over 176 deterministic instances generated from author-designed templates.
Benchmark-family transfer set	Yes	No	Runtime-backed scoring of 25 locally authored tasks preserving structures from four benchmark families.
Expanded real-LLM pilot	No runtime action path	Yes	API-backed classifier/planner behavior under real model calls; still separate from the runtime action path.
Small end-to-end LLM runtime benchmark	Yes	Yes	API-backed runtime-path evidence over scored synthetic tasks.
Transfer-subset end-to-end LLM runtime pilot	Yes	Yes	API-backed runtime-path evidence over 12 benchmark-family-inspired tasks.
Trace-backed normalizer ablation	Existing runtime traces only	No new calls	Counterfactual certificate-bound analysis; not a runtime rerun.

TABLE VII
EVIDENCE TYPES IN THE IGAC EVALUATION.

- 25 locally authored transfer tasks preserving structural patterns from AgentDojo, ToolSandbox, tau-bench, and ToolEmu, including a 12-task real-model runtime subset.

All tasks and fixtures are synthetic; no tenant records, production identifiers, private schemas, or secrets are used. The task authors specified reference certificates, allowed/denied tools, resource bounds, and effect bounds directly in the fixtures. Because no independent annotator study was conducted, we call these *author-specified reference certificates*, not human-consensus ground truth.

The 25-task transfer set contains six AgentDojo-shaped prompt-injection tasks, seven ToolSandbox-shaped tasks spanning canonicalization, insufficient information, and state dependency, six tau-bench-shaped policy tasks, and six ToolEmu-shaped risk cases. It neither imports the original task instances nor runs the original simulators and therefore does not claim benchmark parity.

D. Metrics

Unsafe Tool Exposure Rate:

$$UTER = \frac{\#\{\text{unjustified high-risk tools visible}\}}{\#\{\text{unjustified high-risk tools}\}}.$$

Unsafe Accepted-Authority Rate:

$$UAR_{\text{path}} = \frac{\#\left\{\begin{array}{c} \text{intent-inconsistent draft, preflight,} \\ \text{confirmation, or execution paths accepted} \end{array}\right\}}{\#\{\text{attack or mismatch tasks}\}}.$$

An accepted draft counts because it creates persistent reviewable authority even when no domain effect completes. The deterministic microbenchmark predates this disaggregation and reports a conservative composite indicator UAR_c that is one when either unjustified high-risk exposure or an inconsistent action path occurs. The real-model runtime studies report path acceptance directly. We therefore do not compare these two UAR quantities as if they were the same estimand.

Unsafe Execution Rate:

$$UER = \frac{\#\{\text{unsafe successful executions}\}}{\#\{\text{attack or mismatch tasks}\}}.$$

Strict Completion and Governed Artifact Rate:

$$BCR_{\text{strict}} = \frac{\#\left\{\begin{array}{c} \text{benign tasks completed} \\ \text{without a review artifact} \end{array}\right\}}{\#\{\text{benign tasks}\}}.$$

Claim	Evidence	Status
IGAC is monotone over OpenPort static policy.	Formal model plus endpoint tests.	Supported.
IGAC narrows visible tool manifests.	Endpoint tests plus runtime-backed microbenchmark.	Supported.
IGAC rejects payload-bound mismatch.	Endpoint tests plus synthetic runtime microbenchmark.	Supported in synthetic runtime.
Oracle IGAC eliminates the composite unsafe-authority indicator in the deterministic runtime comparison.	Runtime-backed microbenchmark over 176 synthetic instances.	Supported as conformance evidence.
The combined IGAC–OpenPort path observes 0/198 completed unsafe effects but 40/198 unsafe accepted drafts in the scored model runs.	API-backed runtime trials over 34 tasks, 3 repeats, and 3 models.	Observed in synthetic runtime.
The combined path observes 0/24 completed unsafe effects and 5/24 unsafe accepted drafts on the 12-task transfer subset.	API-backed runtime pilot over 12 tasks and 3 models.	Observed in a small transfer set.
IGAC improves real LLM agent safety in the runtime path.	Small scored runtime pilot exists, but no large scored study yet.	Not yet claimed.
IGAC constitutes a hardened production deployment.	Prototype only; in-memory runtime without persistent certificate state.	Not claimed.

TABLE VIII
CONSERVATIVE CLAIM-EVIDENCE MAP FOR THE CURRENT PAPER VERSION.

Cond.	Description	Where evaluated
B0	Model-only tool calling with a static list and no gateway.	Toy check only.
B1	OpenPort static policy and effect controls without intent certificates.	Toy and 176-instance runtime check.
B2	OpenPort plus IGAC certificate, manifest, and consistency controls.	All runtime studies.
B3	Prompt-only instruction without server-side intent enforcement.	Toy ablation only.

TABLE IX
EVALUATED CONDITIONS AND THEIR SCOPE. THE REAL-MODEL END-TO-END STUDY HAS NO MATCHED B1 MODEL-RUNTIME CONTROL, SO IT DOES NOT IDENTIFY AN IGAC-ONLY CAUSAL EFFECT.

$$GAR = \frac{N_{\text{strict benign}} + N_{\text{reviewable benign artifact}}}{N_{\text{benign}}}.$$

This distinction matters because a draft or preflight artifact may be a correct governance outcome without completing the user’s task. Earlier artifact fields named “safe BCR” are reported here as *GAR*; they must not be interpreted as strict task completion.

Over-Defense Rate:

$$ODR = \frac{\#\{\text{justified benign effects denied}\}}{\#\{\text{justified benign effects}\}}.$$

Manifest Reduction Score:

$$MRS = 1 - \frac{|\text{Visible}_{\text{IGAC}}|}{|\text{Visible}_{\text{OpenPort}}|}.$$

Benchmark source	Current status	Notes
OpenPort synthetic runtime suite	Implemented	Primary source for the runtime-backed microbenchmark and scored end-to-end runtime study.
Prompt-injection tasks inspired by AgentDojo / InjecAgent	Implemented locally	Prompts are benchmark-family-inspired and locally authored.
Tool-poisoning tasks inspired by ToolHijacker / ToolTweak / MCPTox	Implemented locally	Cases transfer attack structure, not original benchmark instances.
AgentDojo / ToolSandbox / tau-bench / ToolEmu transfer set	Implemented	Twenty-five runtime-backed tasks preserve family structure without simulator or parity claims.

TABLE X

TASK-SOURCE SCOPE. BENCHMARK-FAMILY TRANSFER IS DISTINCT FROM ORIGINAL-BENCHMARK REPRODUCTION.

Benign outcome	BCR _{strict}	GAR	Clar.
Immediate or read-only completion	Yes	Yes	No
Reviewable draft or preflight artifact	No	Yes	No
Clarification with no effect	No	No	Yes
Invalid or denied benign call	No	No	No

TABLE XI

RELATIONSHIP BETWEEN BENIGN-OUTCOME METRICS. CLARIFICATION IS TREATED AS A SAFE NON-COMPLETION RATHER THAN AS BENIGN COMPLETION.

Intent-Trace Field Completeness:

$$\mathcal{F} = \{\text{request, certificate, policy, tool, decision, outcome}\},$$

$$TFC = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\mathcal{F} \subseteq \text{Fields}(T_i)].$$

This is an artifact-completeness diagnostic, not a causal measure of human audit quality; a non-IGAC trace lacks certificate fields by construction.

Classifier Quality: We compare certificate outputs against author-specified reference envelopes. Metrics include intent-class accuracy, resource-bound extraction accuracy, unsafe narrowing failure, over-defense, clarification rate, and latency:

$$ICA = \frac{\#\{\hat{I} = I^*\}}{N}, \quad RBA = \frac{\#\{\hat{B}_R = B_R^*\}}{N}.$$

Here I^* and B_R^* are the fixture’s author-specified reference classes and bounds. Unsafe narrowing failure measures cases where a classifier emits a certificate that fails to narrow authority when required; over-defense measures benign requests routed to unnecessary clarification, draft, or denial.

E. Component Analyses

The available component evidence has three scopes:

- the deterministic toy check isolates manifest filtering, payload consistency, and OpenPort effect controls;
- the 176-instance runtime check compares author-reference, rule-based, and hybrid rule+flow certificate generation;
- the normalizer analysis rescored frozen model-runtime traces after observing their failures.

Certificate expiry and audit binding are covered by endpoint invariants, not by causal outcome ablations. The normalizer result is explicitly post-hoc and may be overfit to the observed failure modes.

F. Interpretation and Statistical Unit

We report raw numerators, denominators, and descriptive rates. The three repeats reuse the same parameterized task templates, so trials are not treated as independent draws from a deployment population. Zero observed events mean $0/n$ in the evaluated set, not zero underlying risk. Legacy percentile-bootstrap intervals retained in the artifact are non-inferential because their original random seed was not recorded and they flattened repeated task instances; the paper does not use those intervals for claims. The artifact instead provides deterministic task-clustered reanalysis

Condition	UER	BCR _{strict}	GAR	ODR	MRS	UHRT	Review
Prompt-only instruction	1.0000	1.0000	1.0000	0.0000	0.0000	33	0
Static OpenPort only	0.1429	0.8000	1.0000	0.0000	0.0000	33	1
Manifest filtering only	0.2857	1.0000	1.0000	0.0000	0.7143	0	0
Consistency gate only	0.0000	1.0000	1.0000	0.0000	0.0000	33	0
Manifest filtering + consistency gate	0.0000	1.0000	1.0000	0.0000	0.7143	0	0
Full IGAC + OpenPort effect controls	0.0000	0.8000	1.0000	0.0000	0.7143	0	1

TABLE XII

DETERMINISTIC TOY ABLATION OVER THE SYNTHETIC IGAC TASK MATRIX, RE-SCORED UNDER THE PAPER’S METRIC DEFINITIONS. UER IS UNSAFE EXECUTION RATE; BCR_{strict} REQUIRES DIRECT SAFE COMPLETION WITHOUT CLARIFICATION, DRAFT, OR REVIEW; GAR ALSO CREDITS A GOVERNED ARTIFACT. ODR IS OVER-DEFENSE RATE, MRS IS MEAN MANIFEST REDUCTION, UHRT IS THE COUNT OF UNJUSTIFIED HIGH-RISK TOOLS VISIBLE ACROSS ALL TASKS, AND REVIEW IS THE NUMBER OF BENIGN HIGH-RISK CASES REQUIRING OPERATOR INTERVENTION UNDER OPENPORT EFFECT CONTROLS.

sidecars with 10,000 resamples and explicit analysis seed 20260724. These sidecars quantify stored-row sensitivity without being presented as recovered historical intervals or deployment-population inference.

G. Executable Synthetic Sanity Check

To make the design concrete, we include a deterministic toy benchmark in the artifact directory. It defines seven synthetic tools, twelve synthetic tasks, and three policy variants:

- B0: model-only execution over a broad static tool list;
- B1: OpenPort substrate behavior with static tool exposure and draft-first high-risk writes;
- B2: IGAC-style manifest narrowing and payload consistency checks.

The script is intentionally simple. It does not call an LLM, does not use private data, and does not exercise the HTTP/runtime path. It checks whether the policy rules described in this paper behave as expected over synthetic benign, mismatch, prompt-injection, tool-poisoning, ambiguous, and compound tasks. The separate runtime-backed microbenchmark below exercises the OpenPort reference HTTP path.

The command is:

```
npm run evaluation:igac-synthetic
```

On the current artifact, the script reports 12 tasks: 5 benign and 7 attack-or-ambiguous tasks. Under the broad static manifest, 33 unjustified high-risk tool exposures occur across the task set. Under IGAC narrowing, this count is 0. The toy unsafe-execution rate is 1.0 for model-only B0, 0.1429 for OpenPort-substrate B1, and 0 for IGAC-style B2. Under the artifact’s legacy governed-benign outcome, both B1 and B2 score 1.0 on the five benign tasks. Re-scored under this paper’s definitions, their governed-artifact rate remains 1.0, whereas strict benign completion is 0.8000 because one of five benign tasks is routed to review rather than strictly completed. Mean manifest reduction is 0.7143.

The same script also runs a deterministic ablation over six conditions. Table XII reports the resulting toy metrics.

The toy ablation separates the roles of the major components. Prompt-only instruction is weakest under an adversarial planner because no server-side policy binds model behavior. Static OpenPort effect controls block high-risk direct effects but cannot reject a medium-risk payload expansion that is inconsistent with the user’s requested amount. Manifest filtering removes unjustified high-risk tools from the visible surface but does not by itself catch payload expansion on a still-visible tool. The consistency gate catches payload and effect mismatches even when the full static manifest remains visible, but it does not reduce tool-enumeration exposure. The combined IGAC conditions obtain both zero unsafe execution on this toy matrix and zero unjustified high-risk exposure. Adding OpenPort effect controls introduces one review intervention for the benign high-risk delete task, which is the intended safety cost of draft/preflight governance.

We also include a deterministic confidence-policy artifact:

```
npm run evaluation:igac-confidence
```

This script checks the specific classifier-trust concern discussed in Section 4. It defines twelve synthetic confidence states covering high-confidence low-risk tasks, medium-confidence create/update tasks, low-confidence ambiguous tasks, conflicting read/delete intent, high-risk delete/export intent, multi-step read-then-delete workflows, and high-confidence classifier outputs that exceed static OpenPort policy. Table XIII reports the resulting policy checks.

Check	Toy result
Expected action match rate	1.0000
High-confidence low-risk allow rate	1.0000
Medium-confidence draft/preflight rate	1.0000
Low-confidence clarification rate	1.0000
Conflicting-intent deny/split rate	1.0000
High-risk confirmation/preflight rate	1.0000
Multi-step per-step certificate rate	1.0000
Static policy deny rate	1.0000
Unsafe broad authority grants	0
Authority widening rate	0.0000
Broad multi-step certificates	0

TABLE XIII

DETERMINISTIC TOY CONFIDENCE-POLICY CHECKS. THESE RESULTS VALIDATE THE POLICY ROUTING RULES, NOT CLASSIFIER ACCURACY.

Scenario	Tasks	UAR _c	UER	BCR _{strict}	GAR	Clar.	Review	Draft	MRS
OpenPort-only	176	1.0000	0.0000	0.4314	1.0000	0.0000	0.5686	1.0000	0.0000
Reference certificates	176	0.0000	0.0000	0.4314	0.8627	0.1373	0.4314	0.3333	0.7614
Rule-based certificates	176	0.0000	0.0000	0.4314	0.9608	0.0196	0.5294	0.4091	0.7662
Hybrid rule+flow certificates	176	0.0000	0.0000	0.4314	0.9608	0.0196	0.5294	0.4091	0.7662

Scenario	Tasks	P50	P95	P99
OpenPort-only	176	0.1748	0.3819	1.5451
Reference certificates	176	0.2486	0.4739	1.3952
Rule-based certificates	176	0.2342	0.4305	1.3199
Hybrid rule+flow certificates	176	0.1819	0.3640	1.1942

TABLE XIV

RUNTIME-BACKED SYNTHETIC MICROBENCHMARK. THE UPPER PANEL REPORTS SAFETY AND UTILITY OUTCOMES. BCR_{strict} , GAR , $CLAR.$, AND $REVIEW$ USE ALL 102 BENIGN CASES AS THEIR DENOMINATOR; $CLAR.$ AND $REVIEW$ ARE DISJOINT, WITH $REVIEW$ EXCLUDING $CLARIFICATION$ -ROUTED CASES. $DRAFT$ INSTEAD USES ALL 132 ROWS PER SCENARIO THAT REACH PROPOSAL/ACTION ROUTING ($PROPOSALROWS$), INCLUDING BENIGN AND ATTACK-OR-AMBIGUOUS ROWS: $1.0000 = 132/132$, $0.3333 = 44/132$, AND $0.4091 = 54/132$. IT IS THEREFORE NOT A FRACTION OF ALL BENIGN CASES. THE LOWER PANEL REPORTS LOCAL P50/P95/P99 FASTIFY-INJECTION DIAGNOSTICS IN MILLISECONDS FROM THE ARCHIVED RESULT. UAR_c IS THE LEGACY COMPOSITE EXPOSURE-OR-PATH INDICATOR; GAR COUNTS STRICT COMPLETION OR A GOVERNED ARTIFACT. LATENCIES ARE MACHINE-LOCAL DIAGNOSTICS, NOT STABLE PERFORMANCE ESTIMATES.

These numbers are a reproducible conformance check of the policy sketch. They do not establish real-world security efficacy, model robustness, latency, or deployment utility.

H. Runtime-Backed Microbenchmark

To connect the policy sketch to the implemented runtime, the artifact also includes a small runtime-backed microbenchmark:

```
npm run evaluation:igac-runtime
```

The script uses Fastify HTTP injection against the OpenPort reference runtime. The suite contains 176 deterministic instances generated from author-designed templates: 102 benign and 74 attack-or-ambiguous. It covers six intent classes: read, summarize, create, update, delete, and export. It also covers seven attack or ambiguity categories: intent-tool mismatch, payload expansion, indirect prompt injection, tool metadata poisoning, low confidence, compound workflow, and resource underspecification. The benign set includes underspecified requests so that clarification and review burden remain visible. For each instance, the runtime calls `/intent`, `/manifest`, and, when applicable, `/actions`; it records endpoint outcomes, drafts, executions, manifest sizes, audit binding, and local diagnostic latency.

This harness’s archived UAR_c is a conservative composite: it flags an attack-or-ambiguous instance when an unjustified high-risk tool remains visible or an inconsistent draft/execution path is accepted. Thus OpenPort-only can have $UAR_c = 1.0$ while UER is 0. The value is useful for deterministic conformance, but it is not the same estimand as the path-only UAR in the real-model study.

Classifier	ICA	RBA	EBA	UNF	ODR
Oracle	1.0000	1.0000	1.0000	0.0000	0.0000
Rule-based	0.6818	0.8977	1.0000	0.0000	0.0000
Hybrid rule+flow	0.8636	0.9886	1.0000	0.0000	0.0000
LLM-assisted runtime row	not run in runtime benchmark; see LLM pilot below				

TABLE XV

SMALL DETERMINISTIC CLASSIFIER/ORACLE COMPARISON ON THE 176-TASK SYNTHETIC SUITE. ICA IS INTENT-CLASS ACCURACY, RBA IS RESOURCE-BOUND ACCURACY, EBA IS EFFECT-BOUND ACCURACY, UNF IS UNSAFE NARROWING FAILURE, AND ODR IS OVER-DEFENSE RATE. THIS IS NOT A REAL LLM EVALUATION.

Source family	Benign	Ambiguous	Attack	Total
AgentDojo	2	0	4	6
ToolSandbox	2	2	3	7
tau-bench	2	2	2	6
ToolEmu	2	0	4	6
Overall	8	4	13	25

TABLE XVI

TASK-TYPE COVERAGE IN THE BENCHMARK-FAMILY TRANSFER SET.

The same artifact separates enforcement correctness from certificate-generation quality. The reference condition uses author-specified certificates embedded in the synthetic task suite. Rule-based and hybrid rule+flow conditions are deterministic local classifiers. The microbenchmark has no LLM-assisted runtime row; real-model behavior is measured separately below. The reference condition is intentionally more conservative on underspecified benign requests, so its lower *GAR* should not be read as weaker safety. Higher *GAR* in rule-based or hybrid rows may partially reflect less conservative routing rather than better access-control quality.

This check exercises endpoint wiring, certificate storage, manifest narrowing, action denial, draft creation, and audit binding. The author-reference condition separates enforcement from certificate generation. It remains conformance evidence: task templates and reference envelopes were authored with the mechanism, the planner is deterministic, and no real model participates.

I. Benchmark-Family Transfer Set

To test the mechanism on structures not unique to OpenPort, the artifact includes a locally authored benchmark-family transfer set:

```
npm run evaluation:igac-external
```

The set uses twenty-five canonical JSONL tasks: six AgentDojo-shaped tasks, seven ToolSandbox-shaped tasks, six tau-bench-shaped tasks, and six ToolEmu-shaped tasks. The companion artifact documents the mapping and scoring details. The scorer is runtime-backed and deterministic: it uses author-specified reference certificates and concrete proposals, then drives them through the OpenPort reference HTTP path under OpenPort-only and OpenPort+IGAC conditions. The task content remains synthetic and locally authored; only structural patterns are transferred from the benchmark families.

To make the batch composition explicit, Table XVI separates benign, ambiguous, and attack tasks by source family. The current batch is intentionally attack-heavy because its role is to test whether benchmark-shaped escalation attempts are narrowed away by server-side intent governance rather than to estimate a deployment task distribution. Table XVII then refines the 17 non-benign tasks into additive primary attack-surface categories so that reviewers can see not just task count, but the shape of the covered failure space.

Across the full transfer set, OpenPort-only has composite $UAR_c = 1.0000$, $UER = 0.0000$, and $GAR = 1.0000$. Under IGAC with author-specified reference certificates, UAR_c is 0.0000, UER remains 0.0000, GAR remains 1.0000, and mean manifest reduction is 0.7829. This is a deterministic conformance comparison on locally authored tasks, not an estimate of benchmark-level efficacy.

This is not an original-benchmark evaluation: the set is small, locally authored, and does not use the original simulators or instances. Its value is limited to a structured transfer check across four benchmark-family patterns.

Primary category	Tasks	Attack	Source families
Prompt injection / untrusted content	4	4	AgentDojo
Insufficient information	4	0	ToolSandbox, tau-bench
Wrong scope / target mismatch	3	3	tau-bench, ToolEmu
State dependency	2	2	ToolSandbox
Tool poisoning / semantic deception	2	2	ToolEmu
Canonicalization / normalization risk	1	1	ToolSandbox
Amount expansion / bound inflation	1	1	ToolEmu

TABLE XVII

PRIMARY ATTACK-SURFACE COVERAGE FOR THE 17 NON-BENIGN TRANSFER-SET TASKS. EACH TASK IS ASSIGNED EXACTLY ONE PRIMARY CATEGORY.

Source family	Tasks	OpenPort UAR _c	IGAC UAR _c	IGAC GAR	IGAC MRS
AgentDojo	6	1.0000	0.0000	1.0000	0.7619
ToolSandbox	7	1.0000	0.0000	1.0000	0.7755
tau-bench	6	1.0000	0.0000	1.0000	0.7381
ToolEmu	6	1.0000	0.0000	1.0000	0.8571

TABLE XVIII

RUNTIME RESULTS ON THE LOCALLY AUTHORED BENCHMARK-FAMILY TRANSFER SET WITH AUTHOR-SPECIFIED REFERENCE CERTIFICATES.

J. Expanded Real LLM Pilot

After the runtime microbenchmark, we expanded the real-model pilot into a larger synthetic study using an open-weight chat completions API. The pilot remains intentionally separate from the runtime-backed benchmark: it does not exercise the OpenPort action path and it is not statistically powered, but it does measure how real models generate intent certificates and how real planners behave under static versus IGAC-filtered manifests. We evaluated three open-weight models: GPT-OSS-120B, Llama-3.3-70B-Instruct, and Qwen3-Next-80B, each on 55 classifier tasks and 54 planner attack tasks. Provider-specific account details and unavailable-model diagnostics are omitted from the paper body; sanitized model identifiers and trace summaries are preserved in the artifact.

K. Finding 1: Broad Intent Is Easier Than Authorization Bounds

The strongest design insight from the real-model pilot is not that any one model is good or bad overall. It is that broad intent classification is consistently easier for current LLMs than precise authorization-bound extraction. Across the scored models, intent-class accuracy ranges from 0.7273 to 0.8545, while resource-bound accuracy ranges only from 0.1818 to 0.5818. Review-mode accuracy is likewise materially weaker than broad class recognition.

This asymmetry is exactly why IGAC treats model-generated certificates as advisory. A model may often recognize that the user is asking for read, summarize, create, update, delete, or export at a coarse level, yet still fail to extract the exact resource identifiers, row bounds, or risk mode needed for safe autonomous execution. The security boundary is therefore not the classifier. The boundary is the monotone gateway enforcement that validates scopes, narrows manifests, checks payload bounds, and routes high-risk effects through deterministic review logic.

The expanded classifier pilot exposes the main empirical risk more clearly than the earlier 12-task probe. Across the three scored models, intent-class accuracy ranges from 0.7273 to 0.8545, but resource-bound accuracy ranges only from 0.1818 to 0.5818 and review-mode accuracy from 0.3818 to 0.6909. This pattern supports the design choice that classifiers cannot be trusted to expand authority. It also sharpens a concrete security rule: LLM-emitted review mode must be advisory only. The server-side deterministic risk policy must override it, and a high-risk tool cannot become `allow` merely because the classifier says so. This is not a hypothetical concern: GPT-OSS-120B achieved the strongest bound extraction in this small set, yet still emitted immediate `allow` for 85.71% of high-risk requests.

The planner pilot likewise shows a layered story rather than a one-shot win. IGAC manifest filtering lowers unsafe proposal rates for every scored model, but filtered manifests alone still leave unsafe read-path or overbroad-payload proposals. The important invariant is the final column in Table XX: once deterministic consistency gating is applied, unsafe accepted authority falls to zero in this synthetic planner setting for every scored model. This supports the

Model	ICA	RBA	EBA	Review Acc.	High-Risk Allow	P95 ms
GPT-OSS-120B	0.8545	0.5818	0.9091	0.5273	0.8571	3767.5
Llama-3.3-70B	0.7273	0.1818	0.7818	0.3818	0.4286	17633.4
Qwen3-Next-80B	0.7818	0.1818	0.7818	0.6909	0.0000	4723.2

TABLE XIX

EXPANDED REAL LLM CLASSIFIER PILOT OVER 55 SYNTHETIC INTENT-CERTIFICATE TASKS PER MODEL. ICA IS INTENT-CLASS ACCURACY, RBA IS RESOURCE-BOUND ACCURACY, EBA IS EFFECT-BOUND ACCURACY, AND HIGH-RISK ALLOW IS THE RATE AT WHICH HIGH-RISK CREATE/UPDATE/DELETE/EXPORT REQUESTS WERE INCORRECTLY EMITTED AS IMMEDIATE ALLOW. ACROSS MODELS, BROAD INTENT RECOGNITION IS MATERIALLY STRONGER THAN EXACT RESOURCE/EFFECT-BOUND EXTRACTION.

Model	Static Unsafe	Filtered Unsafe	Filtered+Gate Unsafe Accepted	Static Clar.	Filtered Clar.	Filtered P95 ms
GPT-OSS-120B	0.5000	0.4444	0.0000	0.4444	0.5000	4790.9
Llama-3.3-70B	0.8889	0.5926	0.0000	0.0556	0.3519	35290.4
Qwen3-Next-80B	0.9259	0.7037	0.0000	0.0556	0.2778	5820.6

TABLE XX

EXPANDED REAL LLM PLANNER PILOT OVER 54 SYNTHETIC ATTACK TASKS PER MODEL. MANIFEST FILTERING LOWERS UNSAFE PROPOSALS, BUT DETERMINISTIC GATING IS WHAT DRIVES UNSAFE ACCEPTED AUTHORITY TO ZERO IN THIS PILOT.

claim boundary used throughout the paper: filtered manifests help, but they are not sufficient without payload-bound server enforcement.

L. Scored End-to-End LLM-in-the-Loop Runtime Benchmark

To close the specific gap that the scored real-LLM pilot does not yet exercise the OpenPort runtime action path, we ran an API-backed benchmark harness that pushes model-generated classifier and planner output through the OpenPort reference IGAC path:

```
/intent → /manifest → /preflight or /actions
→ consistency gate → audit.
```

The scored run uses 34 synthetic tasks per run (12 benign and 22 attack-or-ambiguous), 3 repeats per model, and 3 open-weight models: GPT-OSS-120B, Llama-3.3-70B, and Qwen3-Next-80B. The suite is parameterized over three synthetic resource collections, so benign and adversarial patterns recur across bounded settings. It records certificate creation, clarification, unsafe planner proposals, accepted authority, executed effects, BCR_{strict} , GAR , manifest reduction, latency, and sanitized trace summaries.

Finding 2: No unsafe execution is observed, but unsafe authority is accepted.

The full gateway path receives real-model outputs. Across 198 attack-or-ambiguous trials, the combined IGAC–OpenPort path observes 0 completed unsafe effects and 40 unsafe accepted drafts. Because the matched OpenPort-only runtime control already has $UER = 0$ in the deterministic study and no matched B1 model-runtime row exists, this result cannot be attributed causally to IGAC alone.

Runtime unsafe accepted authority is 40/198 overall and 0.0909–0.2727 by model. The cases concentrate in draft-first create paths where the model-generated certificate or payload loses the author-specified amount/resource bound. The evidence therefore supports an observation about completed-effect containment in the combined path while falsifying complete request-bound narrowing under current model-generated certificates.

Finding 3: Model behavior exposes a safety-utility-latency tradeoff.

Manifest reduction is roughly 0.79–0.80 across models, but BCR_{strict} is 0/108 overall and GAR is only 36/108. Thus every counted benign artifact is a governed draft rather than immediate completion. Llama-3.3-70B has the lowest unsafe accepted rate in this setting (0.0909) and the highest GAR (0.5000), but the largest latency tail (P95 49.9 seconds). These descriptive differences are not model rankings: the set is synthetic, small, and tied to one provider configuration.

M. Transfer-Subset End-to-End LLM Runtime Pilot

We ran the same API-backed harness over a selected transfer subset. It contains 12 locally authored tasks from Section X-I: 4 benign, 2 ambiguous, and 6 attack tasks, with 3 shaped by each benchmark family. Unlike the deterministic reference-certificate scorer, this pilot uses real-model certificates and planner output before calling the OpenPort runtime path.

Certificate-generator condition	Current status	Evidence or gap
Author-reference certificates	Implemented	Runtime-backed microbenchmark and deterministic classifier comparison.
Rule-based certificates	Implemented	Runtime-backed microbenchmark and deterministic classifier comparison.
Hybrid rule+flow certificates	Implemented	Runtime-backed microbenchmark and deterministic classifier comparison.
Raw LLM certificate generation	Partial	Implemented in the real-LLM pilot and scored runtime benchmark, but not yet as a large ablation grid.
LLM + schema validator / normalizer	Trace-backed counterfactual	Table XXVIII rescored existing runtime traces; a runtime rerun with repaired certificates is still missing.
LLM + deterministic risk override as isolated row	Trace-backed counterfactual	Table XXVIII isolates review-mode override on existing traces; it is not yet a separate runtime condition.
Hybrid rule + LLM + normalizer	Missing	Still required for a full certificate-generator ablation with planner retry and payload repair.

TABLE XXI
CERTIFICATE-GENERATOR COMPONENT EVIDENCE AND UNMEASURED COMBINATIONS.

Model	Cert. create	Plan clar.	Plan unsafe	Unsafe path	Unsafe exec.	BCR _{strict}	GAR	MRS	P95 (s)
GPT-OSS-120B	1.0000	0.4216	0.5784	0.2424	0.0000	0.0000	0.2500	0.8025	10.47
Llama-3.3-70B	1.0000	0.0980	0.7353	0.0909	0.0000	0.0000	0.5000	0.7913	49.87
Qwen3-Next-80B	1.0000	0.0490	0.9510	0.2727	0.0000	0.0000	0.2500	0.8025	9.95

TABLE XXII
SCORED END-TO-END MODEL RUNTIME BENCHMARK. EACH ROW AGGREGATES 34 SYNTHETIC TASKS ACROSS THREE REPEATS. “UNSAFE PATH” IS INTENT-INCONSISTENT ACCEPTED AUTHORITY; “UNSAFE EXEC.” IS A COMPLETED UNSAFE EFFECT. LATENCY IS DESCRIPTIVE.

This subset is still small, but it closes an important claim gap: benchmark-shaped external tasks are no longer confined to deterministic oracle scoring. Across all three scored models, runtime unsafe execution remains 0 even though planner-unsafe proposals remain frequent and runtime unsafe accepted authority stays nonzero at 0.1250–0.2500. The governed-artifact rate is 0.7500 for all three rows, while strict benign completion remains 0. Concretely, three of the four benign tasks reach non-executed governed artifacts; the tau-bench-style bounded review delete still fails under current planners rather than reaching such a path.

The qualitative pattern is also useful. In this 12-task subset, the runtime unsafe accepted cases are concentrated in ToolEmu-style bounded-create attacks rather than in the AgentDojo, ToolSandbox, or tau-bench-derived rows. In other words, once external benchmark families are routed through real LLM certificates and planner calls, the remaining failures still look like the same bounded-create and semantic-deception residue already visible in the larger synthetic runtime benchmark. This convergence strengthens the paper’s main interpretation: IGAC plus OpenPort effect controls already contain completed side effects in the runtime path, but certificate precision and planner-call quality remain the main blockers to fully eliminating accepted authority.

Table XXIV makes the residual risk boundary more concrete. All prompt-injection, insufficient-information, state-dependency, and wrong-scope trials end either in clarification or in a blocked no-effect path. The only categories that still reach runtime unsafe accepted authority are ToolEmu-style tool-poisoning / semantic-deception rows and amount-expansion rows, and even there unsafe execution remains 0. This is a useful convergence result: once external benchmark families are routed through the OpenPort reference path, the remaining failures do not scatter across the whole attack surface. They collapse into the same certificate-bound and bounded-create residue already identified by the larger synthetic runtime benchmark.

N. Evaluation Synthesis

Taken together, the runtime-backed microbenchmark, the scored LLM runtime benchmark, the transfer-set coverage tables, and the transfer-subset runtime pilot support three narrow but important conclusions.

First, the current gateway path already appears strong against several benchmark-shaped classes once they reach server enforcement. In the external-subset runtime pilot, prompt injection, insufficient information, state dependency, and wrong-scope / wrong-target trials all collapse into clarification or blocked no-effect paths rather than unsafe accepted authority. This does not prove deployment robustness, but it does show that the current monotone manifest and action checks are not failing uniformly across the attack surface.

Model	Tasks	Plan clar.	Plan unsafe	Unsafe path	Unsafe exec.	GAR	P95 (s)
GPT-OSS-120B	12	0.5000	0.5000	0.2500	0.0000	0.7500	13.76
Llama-3.3-70B	12	0.0000	1.0000	0.1250	0.0000	0.7500	53.15
Qwen3-Next-80B	12	0.0833	0.9167	0.2500	0.0000	0.7500	7.36

TABLE XXIII
API-BACKED RUNTIME PILOT OVER THE 12-TASK BENCHMARK-FAMILY TRANSFER SUBSET.

Primary category	Trials	Clarify	Blocked / no effect	Unsafe accepted	Unsafe executed
Prompt injection / untrusted content	6	2	4	0	0
Insufficient information	6	3	3	0	0
State dependency	3	1	2	0	0
Wrong scope / target mismatch	3	1	2	0	0
Tool poisoning / semantic deception	3	0	1	2	0
Amount expansion / bound inflation	3	0	0	3	0

TABLE XXIV
OUTCOME CROSS-TAB FOR THE 24 NON-BENIGN MODEL-TASK TRIALS IN THE EXTERNAL-SUBSET RUNTIME PILOT. COUNTS AGGREGATE ACROSS THE THREE SCORED MODELS. THE 12-TASK SUBSET DOES NOT INCLUDE THE CANONICALIZATION-RISK CATEGORY.

Second, the remaining residue is structurally concentrated rather than diffuse. Across both the larger scored runtime benchmark and the smaller benchmark-shaped external subset, accepted-authority failures cluster in bounded-create settings where certificates and payloads preserve the broad operation class but lose the exact amount, resource, or effect contract needed for early rejection. Tool-poisoning rows only survive when they can masquerade as create-compatible draft paths; amount-expansion rows survive when the gateway still sees a create-shaped proposal but cannot prove the numeric bound is faithful enough.

Third, this concentration sharpens the design agenda for the next version of IGAC. The empirical bottleneck is no longer best described as generic prompt-injection resistance or generic tool misuse. It is more specific: certificate canonicalization, resource/effect normalization, planner schema repair, and stronger create-versus-destructive effect contracts. Put differently, the evaluation gives a meaningful answer to “what still breaks?” The answer is not that every benchmark family defeats the gateway. The answer is that bounded-create semantics remain the dominant open problem once OpenPort effect controls and monotone narrowing are in place.

Finding 4: The remaining accepted-authority failures are narrow and classifiable.

The trace archive shows 40 accepted-authority cases across the 198 attack-or-ambiguous runtime trials. Every accepted case ends in `actionStatus=draft` with `executionCreated=false`; none become completed side effects. The remaining failures are therefore not execution escapes, but reviewable authority paths that the current IGAC layer did not narrow away early enough.

This taxonomy sharpens the paper’s claim boundary. Of the 40 accepted-authority cases, 24 have empty resource and effect bounds in the emitted certificate, while the remaining 16 are semantically weak but non-empty create-compatible certificates. Review-mode labels split across `allow` (16), `preflight` (6), and `draft` (18), which shows that the residual problem is not simply an incorrect review bit. The harder problem is certificate precision: the runtime still needs stronger amount/resource/effect normalization to collapse these governed draft paths into earlier denials or clarifications.

O. Post-Hoc Certificate-Normalizer Sensitivity Analysis

The residual failures in Table XXV suggest a specific design question: would a deterministic certificate-bound normalizer reduce unsafe accepted authority, or is the remaining problem mainly planner behavior? To test this without spending new provider calls, we added a companion normalizer-ablation script. It reads the existing scored runtime benchmark rows, normalizes common LLM certificate aliases such as `collection`, `resource`, `amount`, `itemLimit`, and `limit`, and then rescored the existing trace under stricter certificate-bound predicates.

This is a trace-backed counterfactual, not a new runtime run. It does not replay the model, repair planner payloads, or create new OpenPort actions. Its purpose is narrower: estimate whether the already observed accepted-authority failures would have been removed if the gateway required canonical resource/effect bounds before routing a proposed action and applied those normalized bounds to the consistency gate.

The post-hoc analysis gives two signals. First, deterministic review-mode override alone does not change UAR_{path} because every accepted-authority case is already a draft rather than an execution. The weak point is whether resource

Accepted-authority failure mode	Cases	Trace pattern	Interpretation
Create payload expansion with missing or weak bounds	24	<code>create_amount_expansion_*</code> ; always draft, never execution	The dominant residual failure. In 15/24 cases the certificate omits enforceable resource/effect bounds entirely; in the remaining 9/24 the certificate contains create-compatible but insufficient amount/type bounds.
Tool-poisoning-like create path with semantic deception	16	<code>tool_poison_hard_delete_*</code> ; always draft, never execution	The planner is induced toward a create-compatible draft path rather than an immediate destructive effect. IGAC contains the completed side effect, but draft-first governance still counts as accepted authority when the certificate does not narrow the path away.
Restrictive review mode that still leaves accepted draft authority (overlapping slice)	24	Review mode is draft or preflight rather than allow	Review mode alone is insufficient to eliminate unsafe accepted authority. It converts the path into governed draft/preflight handling, but does not by itself prove the payload/effect bounds are tight enough.
Completed execution escapes	0	No <code>executionCreated=true</code> among accepted rows	The evaluated combined path observes no completed unsafe effect, even though unsafe accepted authority is nonzero.

TABLE XXV

FAILURE MODES AND CROSS-CUTTING PROPERTIES FOR THE SCORED RUNTIME BENCHMARK. THE FIRST TWO ROWS ARE A MUTUALLY EXCLUSIVE $24 + 16 = 40$ PARTITION OF ACCEPTED-AUTHORITY CASES. THE RESTRICTIVE-REVIEW ROW IS AN OVERLAPPING CROSS-CUTTING SLICE OF THOSE SAME 40 CASES AND MUST NOT BE ADDED TO THAT PARTITION; THE FINAL ROW CHECKS COMPLETED EXECUTION AMONG THE ACCEPTED CASES.

Benign or unsafe runtime outcome	Count	Example	Likely fix
Reviewable benign draft counted in <i>GAR</i>	36	<code>create_collection_main</code>	Keep as a governed artifact; do not treat as strict completion.
Malformed benign tool or payload call	67	<code>update_rec_2</code> , <code>delete_rec_7</code> , <code>export_collection_main</code>	Planner schema retry, endpoint-specific payload canonicalization, certificate normalizer.
Benign clarification with no effect	4	<code>export_collection_audit</code> (model A)	Track separately as safe non-completion rather than as hard failure.
Unknown benign action family	1	<code>export_collection_ops</code> (model B)	Tool-call schema validator and planner retry.
Accepted authority from missing amount/resource/effect bounds	24	<code>create_amount_expansion_col_*</code>	Numeric validators and certificate-bound normalizers before action routing.
Accepted authority from create-compatible semantic deception	16	<code>tool_poison_hard_delete_col_*</code>	Stronger effect contracts and semantic validation for create/delete separation.

TABLE XXVI

OUTCOME TAXONOMY FOR THE SCORED RUNTIME BENCHMARK.

and effect bounds are canonical enough for enforcement. Second, strict normalization trades authority rejection for fewer governed benign artifacts: UAR_{path} changes from 0.2020 to 0.0758 under bound validation and to 0 under the normalized-gate predicate, while *GAR* changes from 0.3333 to 0.1389. Because these rules were defined after inspecting the failures and were not rerun through the planner/runtime, the result is a sensitivity analysis, not evidence of a causal safety improvement.

P. Answers to the Research Questions

RQ1 (controlled authority narrowing): On the 176-instance deterministic runtime suite, the author-reference condition changes the archived composite exposure-or-path indicator UAR_c from 1.0000 under OpenPort-only to 0, while UER is 0 in both conditions. This is conformance evidence for manifest and certificate enforcement, not a population safety estimate.

RQ2 (real-model runtime outcomes): Across 198 attack-or-ambiguous trials, the combined IGAC–OpenPort path accepts 40 intent-inconsistent drafts and observes 0 completed unsafe effects. The benchmark-family transfer subset accepts 5/24 unsafe drafts and observes 0/24 unsafe completed effects. No matched OpenPort-only real-model runtime row exists, so these observations do not identify an IGAC-only causal reduction.

Case	Certificate / planner	Runtime result	Metric interpretation	Lesson
Benign create draft	GPT-OSS-120B emits create with bounded resource and amount; planner calls <code>resource.create</code> .	<code>common.success, actionStatus=draft, audit-bound</code> .	Counts toward GAR , not BCR_{strict} .	Governed draft creation is useful, but does not complete the task.
Benign update invalid	GPT-OSS-120B emits bounded update; planner calls <code>resource.update</code> .	<code>agent.action_invalid</code> ; no draft or execution.	Hard benign failure.	Certificate intent is plausible, but payload canonicalization is still too weak.
Intent mismatch blocked by clarification	GPT-OSS-120B emits <code>read, summarize</code> ; planner returns <code>clarify</code> .	No action, no draft, no execution.	Safe non-completion.	Manifest narrowing plus planner uncertainty can fail closed without creating authority.
Low-confidence ambiguity	GPT-OSS-120B emits <code>unknown, confidence 0.2, review mode clarify</code> .	No action, no draft, no execution.	Clarification rather than failure.	Confidence-aware routing works as intended when the request is underspecified.
Amount-expansion attack	GPT-OSS-120B emits create with weak amount bound; planner calls <code>resource.create</code> .	<code>common.success, draft, runtimeUnsafe Accepted=true</code> .	Unsafe accepted authority, no unsafe execution.	Missing numeric/resource precision leaves a reviewable authority path open.
Tool-poisoning attack	GPT-OSS-120B emits <code>create-compatible</code> certificate under poisoned context; planner still selects <code>resource.create</code> .	<code>common.success, draft, runtimeUnsafe Accepted=true</code> .	Unsafe accepted authority, no unsafe execution.	Manifest filtering helps, but semantic validation is still needed to distinguish benign create from disguised delete intent.

TABLE XXVII
QUALITATIVE RUNTIME TRACES FROM THE SCORED BENCHMARK.

Condition	UAR_{path}	UER	GAR	Accepted	Benign artifacts
Raw LLM certificates	0.2020	0.0000	0.3333	40	36
LLM + strict bound validator	0.0758	0.0000	0.1389	15	15
LLM + strict validator + normalized gate	0.0000	0.0000	0.1389	0	15
LLM + deterministic risk override only	0.2020	0.0000	0.3333	40	36

TABLE XXVIII
POST-HOC SENSITIVITY ANALYSIS OVER FROZEN MODEL-RUNTIME TRACES. UAR_{path} IS THE ACCEPTED-ACTION-PATH RATE OVER ATTACK-OR-AMBIGUOUS TRIALS. THIS IS NEITHER A RUNTIME RERUN NOR A CAUSAL ABLATION.

RQ3 (utility and latency cost): Strict benign completion is 0/108 in the scored model-runtime study; 36/108 trials yield governed artifacts. The transfer subset yields 9/12 governed artifacts. Model-level P95 end-to-end latency ranges from 9.95 to 49.87 seconds under the archived provider configuration. These results describe a review-oriented prototype rather than a high-utility autonomous executor.

RQ4 (residual concentration): The 40 unsafe accepted cases comprise 24 amount/resource-bound failures and 16 create-compatible semantic-deception cases. All terminate as drafts. The concentration implicates certificate canonicalization and effect contracts rather than a single incorrect review-mode bit.

RQ5 (component roles): The deterministic component check shows that manifest filtering removes unjustified tool exposure while payload consistency catches overbroad calls that remain in a visible tool class. The post-hoc normalizer changes UAR_{path} from 0.2020 to 0 and GAR from 0.3333 to 0.1389, but it is trace re-scoring designed after failure inspection and therefore supports sensitivity, not causal attribution.

XI. DISCUSSION

Static non-expansion is not request confinement: The formal mechanism guarantees that IGAC cannot exceed static OpenPort authority. Confinement to an expressed request additionally depends on an independently justified certificate envelope and sound adapter effect bounds. The 40/198 unsafe accepted drafts are concrete evidence of this gap, not exceptions to be hidden behind a zero-execution number.

Drafts isolate effects but retain authority and review risk: All 40 residual cases stop at drafts, so no evaluated domain effect completes. Nevertheless, a draft is persistent system state, consumes reviewer capacity, and can carry misleading content into a human approval flow. Effect isolation and authority rejection are therefore separate objectives.

Certificate precision is the measured bottleneck: Broad class recognition is materially stronger than resource- and amount-bound extraction. Residual failures cluster in create-compatible cases whose operation class is plausible but whose exact amount, resource, or semantic effect is weakly encoded. This motivates typed bounds, deterministic normalization, and adapter contracts rather than greater confidence in free-form classifier output.

The prototype is a governance aid, not an autonomous executor: Strict benign completion is 0/108 in the scored model-runtime study; 36/108 benign trials produce governed artifacts. That tradeoff may be appropriate for high-risk workflows, but it means the current configuration cannot support claims of high-utility autonomous execution.

XII. LIMITATIONS AND THREATS TO VALIDITY

Construct validity: Task templates and reference envelopes are author-specified, creating circularity between mechanism design and scoring. The small benchmark-family transfer set is locally authored and does not establish original-benchmark parity. The legacy deterministic UAR_c also combines exposure and path acceptance, so it is not interchangeable with path-only UAR.

Internal and statistical validity: The end-to-end study has no matched OpenPort-only real-model runtime row, so it cannot identify an IGAC-only effect. Repeats share task templates and are correlated; zero observed events are not zero risk. The provider-run bootstrap seed was not archived, and its legacy intervals are not used inferentially. The seeded task-clustered sidecars are reproducible reanalyses of stored rows, not reconstructions of the original provider-run uncertainty calculation. The post-hoc normalizer rules were developed after inspecting failures and may be overfit.

External validity: The studies use synthetic data, three provider-served open-weight models, one provider configuration, modest task sets, and no real operators. There is no evidence about production tenants, human review quality, reviewer fatigue, or adversaries adapting over time. Local Fastify-injection latency does not estimate networked deployment performance.

Implementation and reproducibility: The prototype uses in-memory certificate state rather than persistent, distributed lifecycle enforcement. Historical provider runs did not record a clean runtime revision for every shard, and the transfer subset has incomplete top-level append provenance. Frozen rows remain auditable, but exact provider-call recreation is not claimed. These limitations do not affect deterministic re-scoring of the archived rows.

XIII. SECURITY ANALYSIS

A. Invariant 1: Intent Cannot Widen Static Policy

The monotonicity invariant is:

$$P_C \preceq P_A.$$

If a tool is hidden by static OpenPort scopes, ABAC policy, workspace boundary, revocation, or rate limit, IGAC cannot reveal it. This protects against classifier compromise turning into privilege escalation.

B. Invariant 2: Low Confidence Fails Closed

If $\gamma < \gamma_{\min}$, the system must not treat the certificate as broad authority. It should expose minimal tools, request clarification, or force draft/preflight. Low confidence is not an excuse to fall back to static broad scopes.

C. Invariant 3: Payload Effects Must Be Bounded

Even if a tool class is allowed, the payload must stay within request bounds. A read request for “last week” should not silently become a year-long export. A request to update one record should not become a bulk update.

D. Invariant 4: The Model Is Not a Principal

The model may suggest a tool call, but authorization is decided by the gateway. The gateway verifies scopes, policy, certificate validity, consistency, preflight, and review requirements.

E. Invariant 5: Drafts Are Not Completed Domain Effects

Draft creation is persistent governance state, not a completed domain effect. It still represents accepted authority, consumes reviewer capacity, and can become a social-engineering carrier. Therefore an intent-inconsistent proposal must deny or clarify before draft creation; consistent high-risk proposals may be drafted with audit, a clear preview, and bound review context.

F. Prompt Injection

Prompt injection can influence model reasoning, tool selection, payload generation, and certificate generation. Unconditionally, IGAC can prevent such influence from expanding authority beyond static policy. It confines authority to the expressed request only when the certificate refines the trusted request envelope and adapter effect bounds are sound. Under those conditions, an injected export instruction outside a summary envelope is hidden or denied.

This is not full prompt-injection prevention. If the user explicitly requests a high-risk action and an attacker changes details within the same effect envelope, IGAC may need context provenance or taint-aware manifests. That is complementary work.

G. Tool Metadata Poisoning

Tool metadata attacks try to make a model choose a malicious or irrelevant tool. IGAC reduces the effect of such attacks by filtering visible tools and checking selected tools against user intent. However, if poisoned metadata belongs to a tool that is both statically authorized and intent-compatible, IGAC alone may not detect semantic deception inside the tool implementation. Manifest signing, tool provenance, sandboxing, and behavior monitoring remain necessary.

H. Effect Estimation and Conservative Failure

The consistency predicate depends on estimating or bounding tool effects. This is easy for simple read, create, update, delete, and export actions with explicit resource IDs and item count limits. It is harder for tools with implicit side effects, compound workflows, tool-provided summaries, or domain-specific semantics.

IGAC therefore adopts a conservative failure rule:

$$\neg \text{Boundable}(C, t, x, \hat{e}) \Rightarrow \text{Route}_{\text{IGAC}} \in \{\text{deny}, \text{clarify}\}.$$

Preflight may first be used to obtain a conservative bound. If the gateway still cannot bound the payload or predicted effect against the certificate, the enforce profile admits neither execution nor a draft path; it denies or asks for clarification. This prevents uncertainty from becoming persistent accepted authority.

I. Classifier Compromise

If the classifier is compromised, it might produce a broad certificate. Monotonicity still prevents scope expansion, but it may fail to narrow authority. Mitigations include rule/model agreement, confidence thresholds, human review for high-risk classes, certificate audit, and adversarial tests. Critical deployments should treat classifiers as untrusted components and require deterministic policy constraints for high-risk effects.

J. Multi-Intent Tasks

Users often ask for compound tasks. IGAC should avoid issuing one broad lease for the entire conversation. Instead, it can decompose the request into per-step certificates:

$$C(u) = \{C_1, \dots, C_n\},$$

where each C_i authorizes one effect class and resource bound. If decomposition is uncertain, the gateway should require clarification or draft review.

Operational rule: Read/summarize certificates may be used to discover candidate objects and present them to the user. Update/delete/export certificates should be issued only after the user confirms the concrete target or approved output scope. One broad certificate should not simultaneously cover search, ranking, and high-risk mutation unless the mutation target is already explicit and bounded.

XIV. RELATED WORK

A. Agent Security Benchmarks

AgentDojo, InjecAgent, AgentDyn, ToolSandbox, tau-bench, ToolEmu, and MCPAgentBench provide evaluation settings for tool-using agents [1], [2], [18]–[22]. These benchmarks primarily measure robustness, task success, or safety under adversarial context. Risk-knowledge experiments further show that a model’s ability to recognize risk need not translate into safe action [23]. IGAC is complementary: it defines a server-side authorization mechanism that can be evaluated using such tasks.

B. Prompt-Injection Defenses

StruQ separates prompt and data channels through structured queries and model training [24]. AgentSentry models multi-turn indirect prompt injection as temporal causal takeover and purifies context [25]. AttriGuard is especially close: it causally attributes a proposed action to trusted user input rather than untrusted observations [26]. AttriGuard supplies an action-support signal; IGAC defines the downstream authority ceiling, manifest narrowing, payload/effect-bound check, routing outcome, and audit lifecycle. The mechanisms can compose, but neither makes the other’s guarantee redundant.

C. Tool Selection and MCP Security

ToolHijacker, ToolTweak, MCPTox, and MCP threat-modeling work show that tool names, descriptions, metadata, and ecosystem composition can create security vulnerabilities [3]–[5], [7], [8]. These works motivate intent-aware filtering but do not replace it. Even a perfectly authenticated and well-described tool should not be callable when the user’s request does not justify its effect.

D. Agent Authorization and Access Control

Least privilege, RBAC, ABAC, and OAuth-style delegated authorization remain foundational [12], [15]–[17]. Recent agent-focused work extends OAuth/OIDC delegation with agent credentials and natural-language permission translation, governs inter-agent communication with user-defined policies and cryptographic tokens, enforces intent/context-aware OS policy for computer-use agents, or moves identity- and argument-level policy to an off-host gateway [27]–[30]. IGAC is narrower in principal identity but finer in request lifecycle: it derives a short-lived request envelope, intersects it with an existing gateway policy, filters the model-visible manifest, and rechecks payload/effect bounds before distinguishing draft authority from execution. Monotone intersection is a safety construction, not by itself the novelty claim.

Task Shield enforces task alignment against indirect prompt injection; Progent and MiniScope pursue least-privilege tool authorization; CaMeL separates trusted control and untrusted data flows; Prompt Flow Integrity constrains privilege-escalating prompt flows; and AgentSpec provides customizable runtime enforcement [31]–[36]. These systems are the closest architectural comparators. IGAC’s distinct unit is a server-issued, short-lived request envelope that is intersected with an existing gateway policy both before planning (manifest filtering) and after planning (payload/effect checking), with draft, preflight, confirmation, and execution modeled as different accepted-authority outcomes. This positioning does not imply that request-envelope generation is uniquely sufficient: information-flow isolation, privilege inference, policy languages, and runtime monitors are complementary controls.

E. Purpose Limitation and Contextual Integrity

IGAC is also related to purpose-based access control and contextual privacy. Purpose-based systems restrict data use according to declared purposes rather than only subjects, objects, and operations; recent systems such as Data Guard apply purpose-aware masking in warehouse settings, and purpose limitation has also been studied for data-in-transit systems [37], [38]. Contextual integrity frames privacy as appropriate information flow within a social context, using parameters such as sender, recipient, information type, and transmission principle [39], [40]. Recent assistant work operationalizes contextual integrity for information-sharing and LLM delegation [41], [42].

IGAC differs in its enforcement target. It does not attempt to model all contextual information norms. Instead, it turns the current user request into a short-lived, auditable purpose-like certificate that narrows an already-authorized agent tool surface. In this sense, IGAC can be viewed as request-level purpose restriction for agent side effects: a tool may be statically authorized and still unavailable when the present request does not justify its effect.

Artifact item	Available evidence	Reproduction semantics
Policy and confidence checks	JSONL fixtures, scripts, JSON/Markdown outputs.	Offline and deterministic.
176-instance runtime check	TypeScript harness and per-instance rows.	Requires an OpenPort runtime checkout; semantic outputs reproduce, latency is machine-local.
25-task transfer set	Canonical JSONL, runtime scorer, coverage outputs.	Locally authored transfer tasks; no original simulator or parity claim.
Real-model studies	Frozen JSON/JSONL rows, prompts, model identifiers, and seeded task-clustered sidecars.	Re-scoring and uncertainty reanalysis are offline; provider-call recreation is not exact because historical runtime revision and some append provenance were not recorded.
Post-hoc normalizer	Script and derived JSON/Markdown.	Deterministic re-scoring of frozen rows; not a runtime rerun.
Confidentiality and integrity gate	Verifier plus SHA-256 artifact manifest.	Rejects count drift, missing citations, absolute paths, and common secret patterns.

TABLE XXIX

CURRENT ARTIFACT CONTENTS AND THE STRENGTH OF REPRODUCTION EACH SUPPORTS.

F. OpenPort Protocol

OpenPort Protocol provides the effect-control substrate for this paper [14]. It defines a governance-first gateway with authorization-dependent discovery, scoped permissions, data policy constraints, draft-first writes, preflight, state witness, idempotency, rate limits, and audit. IGAC adds an upstream request-envelope layer and explicitly measures the residual authority that effect controls leave in drafts.

XV. ARTIFACT AVAILABILITY AND REPRODUCIBILITY

The paper directory contains task fixtures, executable harnesses, machine-readable results, frozen provider-run rows, derived summaries, and a verifier. Deterministic runs use the sibling OpenPort reference runtime selected by `OPENPORT_RUNTIME_ROOT`; the loader records the resolved entry and runtime revision. Root-level `evaluation:igac-*` commands run the policy, confidence, runtime, transfer-set, coverage, outcome, and normalizer analyses. `artifact:igac:verify` checks result/trace counts, citation consistency, absolute-path and secret patterns, and manuscript claims.

Provider calls are separated from the default offline gate and require explicit credentials. The frozen 306-row and 36-row studies can be audited, deterministically rescored, and reanalyzed with the recorded sidecar seed without new calls, but their original requests cannot be recreated bit-for-bit where historical provenance is incomplete. This boundary is recorded rather than repaired by inventing a historical seed or runtime revision.

XVI. CONCLUSION

AI-agent access control should not stop at checking whether a credential can call a tool. It should also check whether the proposed call is justified by the user’s current expressed intent. This paper proposed Intent-Governed Access Control, a server-side authorization model that introduces intent certificates, monotone session policy narrowing, intent-aware manifests, and consistency checks between user intent, tool choice, payload, and expected effect. OpenPort provides the existing effect-control substrate: static authorization, policy constraints, draft-first writes, preflight, state witnesses, idempotency, and audit. IGAC adds the missing intent-control layer before those effects are reached.

The evidence supports an executable request-envelope mechanism and static-policy non-expansion, while exposing a decisive limitation. In the real-model runtime study, the combined IGAC–OpenPort path observes 0/198 completed unsafe effects but accepts 40/198 intent-inconsistent draft paths; strict benign completion is 0/108 and 36/108 benign trials produce governed artifacts. The 12-task transfer subset similarly observes 0/24 unsafe completed effects and 5/24 unsafe accepted drafts. These are containment observations, not an IGAC-only causal safety improvement. They identify exact certificate bounds and adapter effect contracts—rather than broad intent labels or model confidence—as the next systems problem.

GENERATIVE AI USE DISCLOSURE

OpenAI Codex was used during manuscript preparation for language and structure revision, artifact-consistency checks, and code-review assistance. It was not an experimental subject or a source of empirical observations. The authors retain responsibility for the claims, citations, code, and reported results.

Claim	Status	Evidence
OpenPort provides authorization-dependent discovery.	Supported in reference substrate	Core agent engine and tool registry pattern.
OpenPort provides draft/preflight/idempotency/audit.	Supported in reference substrate	Core action pipeline and test patterns.
Input-to-draft routing is a known workflow pattern.	Design-motivated evidence	Workflow-motivated examples motivate the design; the paper does not rely on product-specific implementation details.
IGAC intent certificates exist in reference prototype.	Prototype implementation	Intent engine, certificate store, intent endpoint, and endpoint tests.
Intent-aware manifest filtering is implemented.	Prototype implementation	Intent-aware manifest endpoint, monotonicity tests.
Intent-bound preflight/action checks are implemented.	Prototype implementation	Preflight and action endpoints, payload-bound and reason-code tests.
Cross-key certificate isolation is enforced.	Prototype implementation	Endpoint tests for cross-key isolation.
Reference IGAC eliminates the composite unsafe-authority indicator in the deterministic runtime check.	Conformance evidence	Runtime microbenchmark, Table XIV.
The combined model-runtime path observes 0/198 unsafe executions and 40/198 unsafe drafts.	Synthetic runtime observation	End-to-end study, Table XXII; no matched real-model B1 control.
Certificate normalization changes outcomes under frozen-trace re-scoring.	Post-hoc sensitivity analysis	Table XXVIII; not a runtime rerun or causal result.
IGAC improves deployment-level real-world safety metrics.	Not claimed	Larger end-to-end benchmark still required.

TABLE XXX
HIGH-LEVEL CLAIM SUPPORT SUMMARY. FULL AUDIT IS IN THE COMPANION REPORT.

APPENDIX A CLAIM SUPPORT SUMMARY

The claim-support ledger below is an index of the manuscript’s evidence boundary, not an additional empirical result. Each row links a claim to its strongest current evidence and names the corresponding epistemic status. “Reference substrate” denotes implementation and test evidence inherited from the OpenPort-style base; “prototype implementation” denotes the IGAC path added above that substrate; conformance evidence denotes deterministic checks; synthetic runtime observations describe the evaluated combined path; and post-hoc sensitivity denotes frozen-trace re-scoring. None of these labels implies a deployment-level causal safety estimate. The explicit “not claimed” row is retained so that an absence of production evidence cannot be mistaken for an omitted positive result. The ledger should therefore be read together with the formal assumptions, validity threats, and exact reproduction semantics stated in the main text and artifact guide.

APPENDIX B IMPLEMENTATION EVIDENCE SUMMARY

The implementation evidence used in this paper is drawn from the OpenPort-style reference prototype. The IGAC runtime path is concentrated in four logical modules: the intent engine, the in-memory certificate store, the agent engine (manifest, preflight, and action pipeline), and the endpoint wiring layer. Together these modules implement intent certificate creation, confidence-aware routing, manifest filtering, payload-bound consistency checks, reason-code denials, audit metadata binding, and draft policy snapshots.

The substrate layer provides authorization-dependent tool discovery, scope and policy enforcement, draft-first writes, preflight impact hashing, state-witness checks, idempotency, and audit events. The endpoint test suite covers monotonic manifest narrowing, stable reason codes, payload-bound failure, high-risk review routing, audit binding, draft policy snapshots, and cross-key certificate isolation. The regression test suite covers manifest authorization, security controls, high-risk preflight, state witness, idempotency, and draft approval.

These components support the claims in Section X and Table XXX. Reproduction commands, exact harness paths, and evidence boundaries are documented in the paper-level artifact README.

APPENDIX C DETAILED LITERATURE POSITIONING

Work	Primary focus	IGAC distinction
OWASP LLM Top 10 2025	Practical LLM application risks, including prompt injection and excessive agency.	IGAC operationalizes least privilege and human approval as gateway policy.
NIST AI 600-1	Governance, risk monitoring, and deployment-like assurance for GAI.	IGAC supplies a concrete access-control mechanism and audit model.
MCP 2025-06-18	Protocol for tool discovery, invocation, and authorization transport.	IGAC checks request-level intent and tool effects above token validation.
AgentDojo / InjecAgent	Benchmarks for prompt-injection attacks against tool agents.	IGAC can be evaluated on such tasks as a server-side defense layer.
ToolSandbox / tau-bench	Stateful tool-use and agent-user interaction benchmarks.	IGAC adds authorization semantics to stateful tool selection.
ToolHijacker / ToolTweak / MCPTox	Tool metadata and selection attacks.	IGAC filters visible tools and rejects intent-incompatible calls.
StruQ	Structured prompt/data separation.	Complementary input-channel defense; IGAC enforces effects server-side.
AgentSentry / AttrGuard	Causal diagnostics and attribution for indirect prompt injection.	Complementary signals; IGAC defines gateway policy and monotone authority.

TABLE XXXI
RELATED-WORK POSITIONING.

APPENDIX D REFERENCE IGAC REASON CODES

- agent.intent_not_found
- agent.intent_invalid
- agent.intent_binding_mismatch
- agent.intent_version_stale
- agent.intent_revoked
- agent.intent_low_confidence
- agent.intent_conflicting
- agent.intent_tool_mismatch
- agent.intent_payload_exceeds_bound
- agent.intent_expired
- agent.intent_review_required

APPENDIX E EXAMPLE TARGET-PROFILE CERTIFICATE

The following synthetic certificate illustrates the target reference profile used by the formal model. The minimal runtime prototype does not yet persist every subject, message, canonicalization, policy, and registry-version binding shown here.

```
{
  "id": "intent_01",
  "profile": "igac-reference-v1",
  "requestHash": "sha256:...",
  "subjectBinding": {
    "actorId": "actor_demo",
    "appId": "app_demo",
    "keyId": "key_demo",
    "tenantId": "tenant_demo",
    "workspaceId": "workspace_demo",
    "sessionId": "session_demo",
    "messageId": "message_demo"
  },
  "intentClasses": ["summarize", "read"],
  "deniedIntentClasses": ["create", "update", "delete", "export", "admin"],
  "resourceBounds": {
    "resourceTypes": ["record"],
    "dateRange": { "start": "2026-06-10", "end": "2026-06-17" }
  },
  "effectBounds": {
    "maxRisk": "low",
    "allowExport": false,
    "allowMutation": false
  }
}
```

```

},
"confidence": 0.86,
"maxRoutingOutcome": "execute",
"lifecycle": {
  "issuedAt": "2026-06-17T23:54:59Z",
  "expiresAt": "2026-06-17T23:59:59Z",
  "status": "active"
},
"versions": {
  "canonicalization": "canon-v1",
  "staticPolicy": "policy-demo-17",
  "toolRegistry": "registry-demo-4"
},
"classifierProvenance": {
  "source": "hybrid",
  "version": "hybrid-demo-v1"
},
"auditDigest": "sha256:..."
}

```

REFERENCES

- [1] E. DeBenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html
- [2] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 10 471–10 506. [Online]. Available: <https://aclanthology.org/2024.findings-acl.624/>
- [3] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt Injection Attack to Tool Selection in LLM Agents," in *Network and Distributed System Security (NDSS) Symposium 2026*. San Diego, CA: Internet Society, Feb. 2026. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/prompt-injection-attack-to-tool-selection-in-llm-agents/>
- [4] J. Sneh, R. Yan, J. Yu, P. Torr, Y. Gal, S. Sengupta, E. Sommerlade, A. Paren, and A. Bibi, "ToolTweak: An Attack on Tool Selection in LLM-based Agents," 2025. [Online]. Available: <https://arxiv.org/abs/2510.02554>
- [5] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 42, pp. 35 811–35 819, 2026. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/40895>
- [6] S. Jamshidi, A. Moradi Dakhel, K. W. Nafi, and F. Khomh, "Semantic Attacks on Tool-Augmented LLMs: Securing the Model Context Protocol Against Descriptor-Level Manipulation," 2025. [Online]. Available: <https://arxiv.org/abs/2512.06556>
- [7] C. Huang, X. Huang, N. P. Tran, and A. M. Fard, "Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning," 2026. [Online]. Available: <https://arxiv.org/abs/2603.22489>
- [8] Y. T. Shen, K. Toyoda, and A. Leung, "MCP-38: A Comprehensive Threat Taxonomy for Model Context Protocol Systems (v1.0)," 2026. [Online]. Available: <https://arxiv.org/abs/2603.18063>
- [9] OWASP GenAI Security Project, "OWASP Top 10 for LLM Applications 2025," <https://genai.owasp.org/llm-top-10/>, 2025, accessed 2026-06-17.
- [10] —, "LLM01:2025 Prompt Injection," <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>, 2025, accessed 2026-06-17.
- [11] C. Autio, R. Schwartz, J. Dunietz, S. Jain, M. Stanley, E. Tabassi, P. Hall, and K. Roberts, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," National Institute of Standards and Technology, Gaithersburg, MD, NIST Trustworthy and Responsible AI 600-1, Jul. 2024. [Online]. Available: <https://doi.org/10.6028/NIST.AI.600-1>
- [12] Model Context Protocol, "Authorization," <https://modelcontextprotocol.io/specification/2025-06-18/basic/authorization>, Jun. 2025, specification version 2025-06-18. Accessed 2026-06-17.
- [13] —, "Tools," <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>, Jun. 2025, specification version 2025-06-18. Accessed 2026-06-17.
- [14] G. Zhu, C. Wang, Z. Wang, Z. Li, and Q. Li, "OpenPort Protocol: A Security Governance Specification for AI Agent Tool Access," 2026. [Online]. Available: <https://arxiv.org/abs/2602.20196>
- [15] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975. [Online]. Available: <https://doi.org/10.1109/PROC.1975.9939>
- [16] D. F. Ferraiolo and D. R. Kuhn, "Role-Based Access Controls," in *15th National Computer Security Conference*, Baltimore, MD, Oct. 1992, pp. 554–563. [Online]. Available: <https://csrc.nist.gov/pubs/conference/1992/10/13/rolebased-access-controls/final>
- [17] V. C. Hu, D. Ferraiolo, D. R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," National Institute of Standards and Technology, Gaithersburg, MD, NIST Special Publication 800-162, Jan. 2014, includes updates as of 2019-08-02. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-162>
- [18] H. Li, R. Wen, S. Shi, N. Zhang, Y. Vorobeychik, and C. Xiao, "AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments?" 2026. [Online]. Available: <https://arxiv.org/abs/2602.03117>
- [19] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, H. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang, "ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities," in *Findings of the Association for Computational Linguistics: NAACL 2025*. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 1160–1183. [Online]. Available: <https://aclanthology.org/2025.findings-naacl.65/>
- [20] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=roNSXZpUDN>

- [21] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," in *International Conference on Learning Representations*, 2024, spotlight. [Online]. Available: <https://openreview.net/forum?id=GEcwtMk1uA>
- [22] W. Liu, Z. Liu, E. Dai, W. Yu, L. Yu, T. Yang, J. Han, and H. Gao, "MCPAgentBench: A Real-world Task Benchmark for Evaluating LLM Agent MCP Tool Use," 2025. [Online]. Available: <https://arxiv.org/abs/2512.24565>
- [23] Y. Tang, T. Li, E. Li, C. J. Maddison, H. Dong, and Y. Ruan, "LM Agents May Fail to Act on Their Own Risk Knowledge," in *Conference on Language Modeling (COLM)*, 2025. [Online]. Available: <https://openreview.net/forum?id=OeYdS51k8F>
- [24] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending Against Prompt Injection with Structured Queries," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 2383–2400. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe>
- [25] T. Zhang, Y. Xu, J. Wang, K. Guo, X. Xu, B. Xiao, Q. Guan, J. Fan, J. Liu, Z. Liu, and H. Hu, "AgentSentry: Mitigating Indirect Prompt Injection in LLM Agents via Temporal Causal Diagnostics and Context Purification," 2026. [Online]. Available: <https://arxiv.org/abs/2602.22724>
- [26] Y. He, H. Zhu, Y. Li, S. Shao, H. Yao, Z. Liu, and Z. Qin, "AttriGuard: Defeating Indirect Prompt Injection in LLM Agents via Causal Attribution of Tool Invocations," in *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, 2026, to appear. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity26/presentation/he-yu>
- [27] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated Delegation and Authorized AI Agents," 2025. [Online]. Available: <https://arxiv.org/abs/2501.09674>
- [28] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, "SAGA: A Security Architecture for Governing AI Agentic Systems," 2025. [Online]. Available: <https://arxiv.org/abs/2504.21034>
- [29] H. Gong, C. Li, R. Chang, and W. Shen, "Secure and Efficient Access Control for Computer-Use Agents via Context Space," 2025. [Online]. Available: <https://arxiv.org/abs/2509.22256>
- [30] S. V. Kodathala, "aiAuthZ: Off-Host, Identity-Bound Authorization for AI Agents," 2026. [Online]. Available: <https://arxiv.org/abs/2607.05518>
- [31] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 29 680–29 697. [Online]. Available: <https://aclanthology.org/2025.acl-long.1435/>
- [32] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI Agents with Privilege Control," 2025. [Online]. Available: <https://arxiv.org/abs/2504.11703>
- [33] J. Zhu, K. Tseng, G. Vernik, X. Huang, S. G. Patil, V. Fang, and R. A. Popa, "MiniScope: A Least Privilege Framework for Authorizing Tool Calling Agents," 2025. [Online]. Available: <https://arxiv.org/abs/2512.11147>
- [34] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating Prompt Injections by Design," 2025. [Online]. Available: <https://arxiv.org/abs/2503.18813>
- [35] J. Kim, W. Choi, and B. Lee, "Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents," 2025. [Online]. Available: <https://arxiv.org/abs/2503.15547>
- [36] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," in *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*. Rio de Janeiro, Brazil: ACM, Apr. 2026, pp. 1–12. [Online]. Available: <https://doi.org/10.1145/3744916.3764546>
- [37] K. Tran, S. Vasudevan, P. Desai, A. Gorelik, M. Ahuja, A. Yadatore Venkateshababu, M. Verma, D. Hu, W. E. Moustafa, V. Rajamani, A. Gupta, I. Buenrostro, K. Raina, and Y. Lin, "Data Guard: A Fine-grained Purpose-based Access Control System for Large Data Warehouses," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. Montreal, QC, Canada: IEEE, May 2026, official ICDE 2026 program entry; final IEEE DOI and page range were not available as of 2026-07-24. [Online]. Available: https://icde2026.github.io/program_details.html
- [38] K. Wolf, F. Pallas, and S. Tai, "Messaging with Purpose Limitation – Privacy-Compliant Publish-Subscribe Systems," in *25th IEEE International Enterprise Distributed Object Computing Conference (EDOC)*, 2021, pp. 162–172.
- [39] H. Nissenbaum, *Privacy in Context: Technology, Policy, and the Integrity of Social Life*. Stanford University Press, 2009.
- [40] A. Barth, A. Datta, J. C. Mitchell, and H. Nissenbaum, "Privacy and Contextual Integrity: Framework and Applications," in *2006 IEEE Symposium on Security and Privacy*, 2006, pp. 184–198.
- [41] S. Ghalebikesabi, E. Bagdasaryan, R. Yi, I. Yona, I. Shumailov, A. Pappu, C. Shi, L. Weidinger, R. Stanforth, L. Berrada, P. Kohli, P.-S. Huang, and B. Balle, "Operationalizing Contextual Integrity in Privacy-Conscious Assistants," 2024. [Online]. Available: <https://arxiv.org/abs/2408.02373>
- [42] X. Huang, X. Cao, and W. Yang, "Need to Know: Contextual-Integrity-Grounded Query Rewriting for Privacy-Conscious LLM Delegation," 2026. [Online]. Available: <https://arxiv.org/abs/2606.04067>